

ML and LLM lifecycle - principal architect learning plan

Training to production · Study, implement, verify

Print edition · Linked contents and references

Contents

[Lifecycle journey](#)

[Reader guide](#)

[Audience, scope, and learning contract](#)

[Selected pedagogy and session structure](#)

[Evaluate the alternatives](#)

[A-L contract for every delivered module](#)

[Assessment and feedback contract](#)

[Workload and ten-level roadmap](#)

[First two weeks: numbered sessions and evidence](#)

[Module 1: what actually happens when a model is trained](#)

[A - Executive understanding](#)

[B - Mental model](#)

[C - Technical mechanics](#)

[D - Mathematics with a verified step](#)

[E - Implementation decisions](#)

[F - Infrastructure and scale preview](#)

[G - Production architecture](#)

[H - Failure modes](#)

[I - Troubleshooting workflow](#)

[J - Tradeoffs and decision boundaries](#)

[K - Difficult interview question and assessment bank](#)

[L - Independent CPU PyTorch lab](#)

[Five-question quiz - no answers](#)

[Progressive glossary - 10 terms](#)

[Progress log and evidence of learning](#)

[Curriculum briefings and cumulative laboratory roadmap](#)

[Level 1 foundations](#)

[Level 2 deep learning](#)

[Level 3 LLM internals](#)

[Level 4 LLM training and distributed GPUs](#)

[Level 5 fine-tuning](#)

[Level 6 evaluation](#)

[Level 7 deployment](#)

[Level 8 LLMOps](#)

[Level 9 production platform](#)

[Level 10 principal architect](#)

[Engineering numerical models](#)

[Effective batch, steps, and loss normalization](#)

[Weight and training-state memory](#)

[KV cache for a concrete GQA decoder](#)

[Compute, bandwidth, and collective traffic](#)

[Concurrency and Little's Law](#)

[Explicitly fictional illustrative cost model](#)

[Serving alternatives and lifecycle decisions](#)

[Enterprise reference architecture](#)

[Proposed cloud-neutral topology](#)

[Interaction contracts and ownership](#)

[Ten unanswered architecture case prompts](#)

[Capstone request and acceptance rubric](#)

[Topic coverage matrix](#)

[Course resources and verified free-access catalogue](#)

[Corrected course-to-level mapping](#)

[R01 Stanford CS229 Machine Learning public archive](#)

[R02 Neural Networks Zero to Hero including micrograd](#)

[R03 Practical Deep Learning for Coders 2022](#)

[R04 Stanford CS231n Deep Learning for Computer Vision 2025](#)

[R05 Stanford CS224N NLP with Deep Learning Spring 2024](#)

[R06 Hugging Face LLM Course](#)

[R07 Stanford CS336 Language Modeling from Scratch Spring 2026](#)

[R08 GPU MODE profiling and collective communication selections](#)

[R09 Hugging Face smol-course for fine-tuning](#)

[R10 Full Stack Deep Learning 2022](#)

[R11 Full Stack Deep Learning LLM Bootcamp Spring 2023](#)

[R12 DataTalksClub MLOps Zoomcamp](#)

[R13 Introduction to Kubernetes LFS158](#)

[R14 Google The Art of SLOs](#)

[R15 Red Teaming LLM Applications conditional free option](#)

[R16 Google SRE Classroom Distributed ImageServer](#)

[Ranked watch sequence](#)

[Resource verification evidence ledger](#)

[Primary-source register](#)

[Curriculum sources C01-C27](#)

[Module 1 sources S01-S10](#)

[Research evidence summary and validation boundaries](#)

[Research questions resolved](#)

[Evidence classes and dates](#)

[Synthesis validation](#)

[Remaining coverage limits](#)

[Selected approach and actionable next steps](#)

[Follow-up scope](#)

[Personalization questions that do not block the plan](#)

Lifecycle journey

Data -> Representation / tokenization -> Dataset -> Model -> Forward pass -> Loss -> Backpropagation -> Gradients -> Optimizer -> Weight update -> Checkpoint -> Evaluation -> Approval -> Deployment -> Inference -> Monitoring -> Feedback -> Retraining

Tokenization is the language-specific representation stage, not a requirement for scalar regression. Images, audio, tabular inputs, and other modalities need their own representations and preprocessing contracts. Text becomes token IDs before embedding lookup. The arrows name dependencies, not a once-through pipeline: forward through update repeats over batches; validation and checkpointing recur; deployment requires approval; reviewed feedback may propose a new training cycle.

This is a complete learning plan with a fully developed first lesson. The remaining 27 modules have substantive briefings, bounded practical exercises, deliverables, and gates. Ten level labs define the cumulative laboratory roadmap. Only Module 1 contains a full independent runnable program; its runtime has not been exercised. Later lessons, their 28-question banks, and nine-field executable lab expansions will be delivered progressively, not represented as already taught or tested.

Reader guide

- Audience, scope, and learning contract
- Selected pedagogy and session structure
- Workload and ten-level roadmap
- First two weeks: numbered sessions and evidence
- Module 1: what actually happens when a model is trained
 - A-J: understanding, mechanics, mathematics, implementation, and decisions
 - K: 28 unanswered assessment prompts
 - L: full nine-field independent CPU PyTorch lab
 - Five-question quiz, progressive glossary, and progress log
- Curriculum briefings and cumulative laboratory roadmap
 - Levels 1-10, Modules 1-28, and Labs 1-10
- Engineering numerical models
- Serving alternatives and lifecycle decisions
- Enterprise reference architecture and component interactions
- Ten unanswered architecture cases and capstone request
- Topic coverage matrix
- Sixteen-resource course catalogue and ranked watch sequence
- Primary-source register and research evidence summary
- Selected approach, actionable next steps, and follow-up scope

Audience, scope, and learning contract

You already understand enterprise architecture, networking, cloud operations, Kubernetes, and organizational constraints. The missing bridge is the behavior of learned systems: why an update changes a model, why a benchmark can mislead, and why a GPU fleet behaves differently from ordinary stateless services. Start with those mechanisms, not Python syntax or another Kubernetes introduction.

The outcome is evidence-backed design competence, not research mastery or a qualification to train frontier models. Reproduce a small experiment, challenge a memory estimate, distinguish model improvement from application improvement, and defend a production decision. Large-scale training, security assurance, and principal-level judgment also require team experience and specialist review.

Every threshold is an educational acceptance criterion unless explicitly marked as a mathematical invariant. Production thresholds require workload evidence and accountable approval. A favorable chart without a valid comparison is not a pass. A negative result with a sound diagnosis can be a successful deliverable.

Use a separate learning environment and approved compute. Preparation research reported Python 3.11.15 and no PyTorch in the application's virtual environment. No application source, dependencies, or configuration are changed by this plan. Windows supports reading and CPU exercises. Later CUDA/NCCL and serving exercises need a supported Linux environment or remote Linux GPU host. Without multiple GPUs, complete design/replay tracks but leave distributed-performance gates pending.

ARISE-X is an optional later evaluation/AgentOps connection, not a prerequisite, pretraining engine, fine-tuning platform, or claim of production readiness. Existing project documentation offers examples of trajectory evidence and fail-closed gates; executable integrations require a separate current-source verification.

Selected pedagogy and session structure

Evaluate the alternatives

Approach	Strength	Limitation	Selection
Tool-first walkthroughs	Fast endpoint and motivation	Hide leakage, optimizer state, token scheduling, and incompatible defaults	Short demonstrations only after understanding the mechanism
Theory-only study	Mathematical depth	Delays operational feedback and can overinvest in proofs unrelated to decisions	Use targeted derivations tied to observable behavior
Platform-first architecture	Reuses Kubernetes expertise	Treats weights as ordinary assets and quality as uptime	Reconnect platform skills after training/evaluation foundations
Measured concept-first progression	Connects prediction, implementation, failure, and decision	Requires deliberate evidence capture	Selected default with stage gates

Predict an outcome, change one factor, measure intended and unintended effects, then explain discrepancies. Keep a hypothesis ledger recording baseline, controls, data identity, precision, hardware, seed, result, uncertainty, and decision. Read primary documentation narrowly at the relevant decision, not as a memorization exercise. Pin versions and model revisions when implementing labs.

A-L contract for every delivered module

Section	Required teaching content
A Executive understanding	Purpose, business consequence, and developer-to-principal perspectives
B Mental model	A concrete analogy and the limits of that analogy
C Technical mechanics	State, tensor/API contracts, and ordering
D Mathematics	A bounded derivation with explicit assumptions and units
E Implementation decisions	Code-level choices and evidence to inspect
F Infrastructure and scale	Resource model, bottlenecks, and hardware limits
G Production architecture	Component boundaries, ownership, and artifact flow
H Failure modes	Observable symptoms, mechanisms, and prevention
I Troubleshooting workflow	Smallest reproduction and causal inspection order
J Tradeoffs	Alternatives, decision boundaries, and rejected choices
K Assessment	One difficult interview question within the 28-prompt bank
L Practical lab	Objective, architecture, prerequisites, code, explanation, expected output, observation, failures, production equivalent

A module may span several study sessions. The numbered sessions below work through this same A-L contract; they are not twelve new modules. Every future module gets a bounded exercise now and its full A-L lesson when reached. The nine lab fields are expanded into executable instructions at that point. The ten cumulative lab specifications are not a claim of 28 complete runnable labs.

Assessment and feedback contract

Each delivered module has exactly 5 beginner + 5 intermediate + 5 senior + 5 principal + 3 design + 3 troubleshooting + 2 whiteboard prompts, totaling 28, plus a separate five-question quiz. The difficult question is counted once in the principal group. Module 1's complete bank is supplied below; later banks are not supplied or claimed assessed yet. Do not provide answer keys before you attempt the prompts. Architecture cases and the capstone also remain unsolved requests.

When you submit an answer, return: score /10; what was right; what was missed; specific weaknesses; a better answer after the attempt; and the principal-level perspective on evidence, risk, ownership, and alternatives. Score using correctness 4, causal reasoning 2, explicit assumptions/evidence 2, and production

judgment 2. Record the prompt ID and a concrete remediation exercise. A suggested progression gate is an average of 8/10 over the selected review set plus the module's practical gate; missing mandatory integrity evidence cannot be offset by a high average.

Question IDs are scoped to Module 1, for example M1-S1 for a senior question. Evidence IDs use C01-C27 for curriculum primary documents, S01-S10 for first-lesson sources, and R01-R16 for course resources. These namespaces are distinct.

Workload and ten-level roadmap

There are 28 modules across exactly ten levels and 486 core hours. Hours include reading, module exercises, cumulative level labs, and one review; do not add the lab time again. Reserve 20 percent: 97.2 additional hours, 583.2 total, rounded up to about 584 hours for planning. At 8-10 hours weekly, budget approximately 59-73 study weeks or roughly 15-19 calendar months with interruptions. This is adjustable, not a mastery guarantee or deadline.

Level	Modules	Core hours	Weeks at 8-10 hours
L1 Foundations	1 Numerical learning and experimental reasoning	10	1.0-1.25
L2 Deep learning	2 Networks; 3 Optimization; 4 Precision and reproducibility	48	4.8-6.0
L3 LLM internals	5 Tokenization; 6 Transformers; 7 Generation and context	48	4.8-6.0
L4 LLM training and distributed GPUs	8 Pretraining; 9 Distributed training/fabrics; 10 Post-training objectives	56	5.6-7.0
L5 Fine-tuning	11 Full FT/LoRA; 12 QLoRA and adapters; 13 Adaptation decisions	48	4.8-6.0
L6 Evaluation	14 Model/statistical; 15 Application/agent; 16 Safety/drift/release	54	5.4-6.75
L7 Deployment	17 Artifacts/runtimes; 18 Scheduling/performance; 19 Serving/delivery	54	5.4-6.75
L8 LLMOps	20 Lineage; 21 Observability; 22 Feedback/incidents	48	4.8-6.0
L9 Production platform	23 Kubernetes GPU; 24 Security/isolation; 25 Capacity/economics/DR	56	5.6-7.0

Level	Modules	Core hours	Weeks at 8-10 hours
L10 Principal architect	26 Enterprise architecture; 27 Assurance; 28 Capstone	64	6.4-8.0

A weekly pattern is two hours of mechanism reading, four of experimentation, two of interpretation, and up to two of review. Complete levels 1-3 before adaptation, and level 6 before external-user deployment. Held-out checks begin in Module 1; level 6 deepens evaluation rather than introducing it late. Lab margins, dataset sizes, and synthetic costs are educational, not procurement or production rules.

First two weeks: numbered sessions and evidence

Use eight core hours per week. Optional two-hour weekly review buffers bring the schedule to ten without increasing the 486-hour scope. Week 1 covers eight hours of Module 1; week 2 spends two completing Module 1 and six beginning Module 2. Progress depends on evidence, not the calendar. Without an approved CPU lab runtime, use the code-reading track and keep execution-dependent gates pending.

Week/session	Time	Section and bounded task	Expected evidence, not assumed completion
1 / 1	2h	M1 A-C: draw the lifecycle; distinguish parameters, graph, gradients, optimizer; inspect shape contracts	One annotated lifecycle with loops/approval and a four-operation mutation table; predict M1-B1/B2/B3 responses
1 / 2	2h	M1 D: derive both gradients from the two examples; recompute both parameters and MSE; use R02 micrograd selectively	Hand-worked trace, units, and independent arithmetic check; no unseen expected value accepted without derivation
1 / 3	2h	M1 E-F/L1-L4: review a separate environment, read the independent program, predict each assertion; execute only after approval/setup	Environment/version record and captured single-step output if run, otherwise annotated code and explicit execution-pending label
1 / 4	2h	M1 G-J/L5-L9: inspect split/normalization/checkpoint contracts; introduce one disposable shape fault and one resume fault	Split ledger, failing/repaired assertions if run, and training-versus-serving state inventory; list recovery limits
2 / 5	2h	M1 K and quiz: attempt all 28 prompts briefly, whiteboard the two derivations, submit one deeper principal response; record gaps	Answer sheet without prefilled answers, five quiz responses, requested score/10 feedback, and Module 1 gate decision

Week/session	Time	Section and bounded task	Expected evidence, not assumed completion
2 / 6	2h	Begin M2 A-D briefing: compare linear composition with a nonlinear two-layer model; choose the controlled Lab 2 dataset	Shape diagram, exact parameter count, activation rationale, and preregistered linear/MLP comparison
2 / 7	2h	M2 E-F exercise, linked to Lab 2: implement or inspect a small baseline and MLP using the same split/budget	Run manifest and learning curves if executed, otherwise a bounded implementation plan; no invented metrics
2 / 8	2h	M2 H-J exercise: introduce one overfitting condition and inspect held-out errors; request the full M2 lesson next	Error slices, alternative explanations, one revised experiment, and pending M2 K/L expansion checklist

Review buffers: week 1, replay the derivation without reading it; week 2, remediate the lowest-scored Module 1 weakness before proceeding. If Session 5 cannot meet the practical gate, replace Sessions 6-8 with remediation and shift Module 2. No Module 2 assessment result is inferred from starting its exercise.

Module 1: what actually happens when a model is trained

A - Executive understanding

Training changes numerical parameters so a chosen objective becomes smaller on training examples. It does not install factual records into a searchable database, prove that predictions are correct, or authorize deployment. The executable model defines a family of functions. Data, the objective, initialization, and optimization determine which member of that family you obtain.

Use one scalar input, one scalar output, and two trainable numbers: $\hat{y} = wx + b$. You already know the function's form; training estimates its slope and intercept. A neural network replaces this function with a composition of many parameterized operations. The forward/loss/backward/update contract remains.

The success criterion is operational: predict every number in one update, identify which objects mutate, diagnose a wrong gradient, and distinguish an optimization check from evidence of generalization. Derive the numbers before running the lab.

Perspective	Responsibility and evidence
Developer	Tensor contracts, graph connectivity, assertions, executable updates
ML engineer	Objective, split validity, normalization, optimization, evaluation

Perspective	Responsibility and evidence
Platform engineer	Data delivery, memory, reproducible runtimes, recoverable state
Architect	Training/serving boundaries, artifact contracts, gates, rollback
Principal engineer	Evidence for the business decision and operating envelope

No learner proficiency is inferred from reading. The progress log separates covered topics from demonstrated competence.

B - Mental model

Imagine two adjustable knobs, w and b . A forward pass asks what the current settings predict. The loss measures a specified discrepancy. Backpropagation computes local sensitivity: how changing each knob infinitesimally would change that loss. The optimizer chooses a finite adjustment from those sensitivities, its hyperparameters, and possibly its history.

A gradient is not the update. It does not contain a learning rate, and it is not a promise of improvement. A negative derivative means increasing that parameter locally decreases the objective while other parameters are fixed. Subtracting the gradient moves in the locally decreasing direction. Large moves can leave the region where that local description is useful.

Keep three kinds of state separate:

- Persistent learned state: parameters and relevant model buffers
- Temporary computation state: activations, saved tensors, and the current graph
- Training continuation state: gradient buffers, optimizer history, random-number generators, data position, schedules, and progress counters

A batch contains examples processed together. A step usually means one optimizer update. An epoch means one traversal of the designated training set. Accumulation allows multiple microbatches per step; streaming datasets may not have a natural epoch. Always say which counter a graph or checkpoint uses.

In the lab, data is numeric tensors. In language modeling, text becomes token IDs, then embeddings and other representations. Token IDs are discrete identifiers, not differentiable real-valued inputs; selected embedding weights receive gradients. Tokenizer construction and transformer internals come later. CNNs reuse filters across spatial positions; RNNs reuse transition parameters across time. Their shapes and backward details belong in later modules.

C - Technical mechanics

Tensor contracts before algorithms

For batch size $B = 2$, use inputs and targets of shape `[B, 1]`, not one of shape `[B]` and the other `[B, 1]`. Broadcasting that mismatch can produce a `[B, B]` error matrix comparing every prediction with every target. The code can run while optimizing the wrong objective.

`nn.Linear(1, 1)` stores weight shape `[1, 1]` and bias shape `[1]`. Its operation is $XW^T + b$, giving output `[B, 1]`; bias broadcasts across rows. MSE reduces all output elements to a scalar with shape `[]`. With one output per example, averaging elements equals averaging examples. Reconsider that equivalence for multiple outputs, masks, and per-example weights.

Graph construction and leaf parameters

Assigning an `nn.Parameter` to a module attribute registers it as module state; the wrapper normally enables `requires_grad`. Parameters are leaves and have no producing `grad_fn`. Outputs and loss are non-leaf results with graph history when computed in grad mode. Inputs need not require gradients for parameter gradients to be computed.

Eager autograd records operations actually executed and saves needed values. A fresh forward creates a fresh graph. `loss.backward()` starts with $\partial L / \partial L = 1$ for a scalar loss and traverses dependencies in reverse, applying the chain rule and summing shared-use contributions (S01). Reverse-mode autodiff is neither finite differences nor symbolic algebra over the whole program. It applies implemented local derivatives at the current numerical values.

Leaf parameters normally accumulate results into `.grad`. Intermediate gradients are used during backward but not retained in their `.grad` fields; call `retain_grad()` on a chosen non-leaf before backward for inspection. Saved tensors are normally released during backward. Reusing a consumed graph differs from a fresh forward; routinely setting `retain_graph=True` does not fix a broken loop.

Four APIs with different jobs

Operation	What changes	What it does not do
<code>optimizer.zero_grad(set_to_none=True)</code>	Assigned parameter gradients become <code>None</code>	Reset weights or momentum
<code>model(x)</code> and loss construction	Predictions, scalar loss, graph	Apply an optimizer update
<code>loss.backward()</code>	Accumulates parameter gradients	Change ordinary model weights
<code>optimizer.step()</code>	Assigned parameters and optimizer state	Normally clear gradients

Two fresh forwards and backwards without clearing add contributions. This can implement intentional accumulation or an accidental larger update. Clearing between backward and step removes the information the optimizer needs. Clear once at the start of an intended update.

`None` and a zero tensor are not equivalent (S03). Optimizers generally skip a parameter whose gradient is `None`; a zero gradient can still permit momentum or decay to move it. Check for `None` before inspecting a gradient norm. An optimizer knows only the parameters passed to it, not every tensor in a model.

`model.train()` and `model.eval()` select behavior such as dropout and BatchNorm. They do not enable/disable autodiff. `torch.no_grad()` prevents graph recording for ordinary operations, not training-mode behavior. Validation normally needs both `eval()` and `no_grad`. The lone linear layer has no mode-sensitive behavior, but preserving the contract avoids later bugs (S01). Ordinary optimizer updates do not record a new gradient graph; differentiable optimization is an advanced case.

D - Mathematics with a verified step

Forward and loss

Let $x = [1, 2]$, $y = [2, 4]$, $w_0 = 0.5$, $b_0 = 0$, and learning rate $\eta = 0.1$. Both parameters update together from gradients evaluated at the old state.

$$L(w, b) = \frac{1}{B} \sum_{i=1}^B (wx_i + b - y_i)^2.$$

Example	Input	Target	Prediction	Residual	Squared residual
1	1	2	0.5	-1.5	2.25
2	2	4	1.0	-3.0	9.00

$L_0 = (2.25 + 9)/2 = 5.625$. There is no extra factor of one-half. Textbooks sometimes use one; mixing conventions changes gradients. Loss has squared target units, so comparisons across differently scaled targets can mislead.

Backward through each operation

Define $z_i = wx_i + b$, $e_i = z_i - y_i$, and $q_i = e_i^2$. The local derivatives are $\partial L / \partial q_i = 1/B$, $\partial q_i / \partial e_i = 2e_i$, $\partial e_i / \partial z_i = 1$, $\partial z_i / \partial w = x_i$, and $\partial z_i / \partial b = 1$. Multiply along paths and add contributions:

$$\frac{\partial L}{\partial w} = \sum_i \frac{1}{B} (2e_i)(1)x_i = \frac{2}{2} [(-1.5)(1) + (-3)(2)] = -7.5,$$

$$\frac{\partial L}{\partial b} = \sum_i \frac{1}{B} (2e_i)(1)(1) = \frac{2}{2} [-1.5 - 3] = -4.5.$$

The prediction gradient is $[-1.5, -3]$. The second example contributes more to the slope gradient because $\partial(wx_2)/\partial w = x_2 = 2$. Contributions sum because the same parameters serve both rows, not a separate weight per row.

Update and recompute

Plain SGD without momentum or decay applies $\theta_1 = \theta_0 - \eta g_0$:

$$w_1 = 0.5 - 0.1(-7.5) = 1.25, \quad b_1 = 0 - 0.1(-4.5) = 0.45.$$

The new predictions are $[1.70, 2.95]$, and residuals are $[-0.30, -1.05]$:

$$L_1 = \frac{(-0.30)^2 + (-1.05)^2}{2} = \frac{0.09 + 1.1025}{2} = \frac{1.1925}{2} = 0.59625.$$

The original `loss` tensor still contains the pre-update value. Recompute a forward to measure post-update loss. This is a same-batch optimization diagnostic, not validation or test performance.

A finite-difference check approximates $\partial L / \partial w$ with $[L(w + h, b) - L(w - h, b)] / (2h)$. It is useful for tiny diagnostic cases, not a replacement for reverse-mode autodiff over millions of parameters. Too small an h amplifies cancellation; too large introduces approximation error for general nonlinear functions.

Beyond plain SGD

One momentum convention is $v_t = \mu v_{t-1} + g_t$ and $\theta_t = \theta_{t-1} - \eta v_t$, with zero initial velocity, no dampening, and no Nesterov term. Momentum remembers directions, so weights-only resume generally changes the next update. Library conventions matter (S02).

For ordinary Adam without weight decay or AMSGrad, operations are elementwise:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_t = \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

The second moment tracks squared gradients, not centered statistical variance. Bias correction compensates for initial zero moment estimates. AdamW decouples decay from moments:

$$\theta_t = (1 - \eta_t \lambda) \theta_{t-1} - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Here g_t is the data-objective gradient. Traditional coupled Adam decay adds $\lambda \theta$ before forming moments. These are not generally equivalent under adaptive scaling (S04-S05). Neither guarantees monotonic loss on stochastic batches.

For a finite objective $N^{-1} \sum_i \ell_i(\theta)$, uniformly sampled examples give an unbiased mean gradient at a fixed θ if sampling and reduction match the objective. Unequal sampling without correction changes the expectation. Random reshuffling is not conditionally identical to independent sampling at every evolving state. Batch-dependent operations, clipping, variable-length normalization, and filtering need separate analysis. An unbiased gradient is not an unbiased Adam update or proof of production generalization.

E - Implementation decisions

Construct the model on its intended device and dtype before the optimizer. Replacing parameter objects afterward can leave the optimizer attached to obsolete objects. Initialize using `torch.no_grad()` and in-place `fill_`, not `.data` mutations that bypass autograd checks. Fixed values isolate arithmetic from chance.

Use floating targets for regression. Check equal input/output/target shapes, finite loss, expected nonmissing and finite gradients, and actual parameter movement. Changing loss alone cannot identify a wrong target or missing parameter.

The lab uses double precision for arithmetic inspection, then another linear model on different points. It imports no ARISE-X modules, downloads no datasets, needs no GPU, and has no notebook-state dependency. Fit preprocessing on training values only. Validation selects a snapshot; test is scored once after selection. Copy selected state rather than retaining a live `state_dict` reference (S08).

For standardized $z = (x - \mu)/\sigma$, the model is $az + c$. Raw slope and intercept are a/σ and $c - a\mu/\sigma$. Comparing standardized a directly with raw target slope 2 is wrong. Persist the transform, feature order, and expected dtype with weights. Serving reuses the transform; it does not refit it.

F - Infrastructure and scale preview

Memory is not weight size

For P parameters in all-FP32 Adam, a common baseline is 4 bytes for weights, 4 for gradients, and 8 for two moments: $16P$ bytes. One billion parameters means 16 GB decimal, about 14.9 GiB, before activations or overhead. This is an assumption, not a universal training-memory formula.

Activations depend on batch, depth, representation, sequence length, and saved intermediates. Add temporary kernels, allocator reservation, communication buckets, buffers, and checkpoint staging. First-update optimizer-state allocation can fail after a successful forward. FP64 doubles this four-array baseline; plain SGD has fewer arrays. Master weights, low-bit optimizers, and sharding change the ledger. Measure actual allocated/reserved peak memory.

CPU decoding, RAM, storage bandwidth, and host-to-device transfers can starve GPUs. Profile data wait, forward, backward, optimizer, and checkpoints separately. GPU utilization alone cannot locate a bottleneck. Two scalar parameters belong on CPU.

Accumulation with the correct denominator

For K equal microbatches and an additive mean objective, divide each microbatch mean by K , backward each, then step once. With different valid-token counts n_k , equal averaging is wrong for a token-mean objective:

$$L = \frac{\sum_k \sum_{j=1}^{n_k} \ell_{kj}}{\sum_k n_k} = \sum_k \frac{n_k}{N_{\text{valid}}} L_k.$$

Two and eight valid tokens require weights 0.2 and 0.8, not 0.5 each. Mask padding, count valid elements, and reject an all-masked window. Backward summed losses divided by the window total, or accumulate summed gradients and divide once before clipping and stepping. Do not step between microbatches. Handle partial final windows and use the actual denominator for weighted objectives. Large-batch equivalence also requires unchanged parameters and compatible per-example computations; BatchNorm, dropout, rounding, or per-microbatch clipping can break it.

Precision and placement

FP16 gradients can underflow; values can overflow. Autocast chooses operation precision. A gradient scaler scales loss and unscales gradients before inspection or clipping. Nonfinite gradients can skip an update and adjust the scale. Keep scale unchanged through accumulation, unscale once, then clip, step, and update the scaler (S06). Track successful updates separately and align scheduler policy.

BF16 has FP32-like exponent range but fewer significand bits and often avoids FP16-style scaling. It is not immune to rounding, instability, or NaNs. Neither precision mode is exercised by this CPU FP64 lab.

Data parallel replicas combine local gradients before updating. Equal local means can be averaged for equal counts. With unequal token counts and default averaging over R ranks, scaling summed local losses by R/N_{global} before the averaged reduction yields the global token-mean gradient. Custom communication can differ. State sharding partitions storage; tensor/pipeline parallelism partitions computation. These are previews, not distributed implementation claims.

G - Production architecture

Separate the reproducible training job from request serving. Training consumes versioned data/configuration and emits recovery checkpoints, candidates, and evidence. Serving loads an approved inference artifact and preprocessing contract, validates requests, and predicts. It normally needs no labels, backward graph, or optimizer moments.

```
Versioned data + split manifest + configuration
-> train-only preprocessing fit -> training worker -> recovery checkpoint
-> validation selection -> frozen candidate -> independent test evidence
-> approval / registry -> staged serving rollout -> monitored predictions
-> reviewed outcomes and labels -> new data version -> retraining proposal
```

A recovery checkpoint needs model/optimizer state, step/epoch and data cursor, random states, configuration, preprocessing/schema, dataset identity, code revision, and runtime versions. Add scheduler/scaler state when used. Multiworker loading, CUDA RNGs, and partial accumulation need continuation state too. A seed restarts a sequence; an RNG state resumes the current position.

Use integrity checks and atomic publication for durable checkpoints. Test actual restoration, not only writing. The lab's in-memory round trip demonstrates state, not durability, process-crash recovery, or cross-machine bitwise replay. Load only trusted artifacts; `weights_only=True` narrows the deserialization surface but does not make arbitrary files harmless (S08).

Release evidence includes task performance, slices, latency, cost, compatibility, and rollback. Monitor validity, distributions, health, and delayed outcomes. Drift signals investigation, not an automatic retraining benefit. Feedback may be delayed, selection-biased, incorrect, or unsuitable for retention. Review, version, and deduplicate before proposing training data. Deployment and retraining are controlled loops, not automatic consequences of falling loss.

H - Failure modes

Failure	Mechanism	Prevention or evidence
Wrong broadcasting	<code>[B]</code> targets meet <code>[B,1]</code> predictions	Assert equal shapes
Missing gradients	Detach, Python scalar conversion, no-grad forward	Inspect <code>requires_grad</code> , <code>grad_fn</code> , leaf <code>.grad</code>
Stale accumulation	Clearing omitted between intended steps	Track update boundaries
No parameter movement	Wrong optimizer membership or clearing after backward	Inspect identities and before/after values

Failure	Mechanism	Prevention or evidence
Exploding loss	Step size, invalid data, or scaling	Finite checks and gradient/update norms
Misleading validation	Training-batch scoring or all-data preprocessing	Disjoint splits and train-only fitting
Resume divergence	Missing optimizer/RNG/cursor/transform	Compare next batch and update
Growing memory	Retained graphs or graph-connected logs	Detached metrics and reference inspection
Serving mismatch	Feature units/order/transform differs	Versioned input contract

A zero gradient can be correct; `None` may be expected for an unused branch. Diagnose against the intended graph. Lower loss can coexist with leakage or an undesired proxy. Numerical health is necessary, not sufficient.

I - Troubleshooting workflow

Start with the smallest fixed batch and weights. Record shapes, dtype, device, predictions, targets, reduction, and finite checks before changing the optimizer. Reproduce the hand step; a larger model only obscures a failure there.

Inspect gradients immediately after backward and parameters after step. Log gradient and update norms separately. If gradients exist but weights do not move, check learning rate, membership, and precision. Find the first detached conversion; setting `requires_grad=True` on a final disconnected scalar cannot reconnect it.

For NaNs, find the first nonfinite tensor. Invalid operations can poison backward even if outputs are masked later; prevent the invalid operation before recording it (S01). Use anomaly detection temporarily on a small reproduction, not always-on. Compare higher precision when useful.

For resume mismatches, inspect in causal order: configuration/data IDs, transforms, next batch, pre-step weights, forward output, gradients, optimizer buffers, updated weights. Same weights plus different moments are different training states. Isolate this before investigating hardware nondeterminism.

Separate optimization from generalization. Failure to fit a small clean subset suggests implementation/capacity problems. Training fit with held-out failure suggests splits, distributions, capacity, or selection issues. Repeated test-set tuning turns that test into another development set.

J - Tradeoffs and decision boundaries

Choice	Benefit	Cost or boundary
CPU FP64 toy example	Inspectable arithmetic without GPU	Not representative throughput/memory
Larger batch	Parallel work, often less sampling noise	Activations and altered optimization

Choice	Benefit	Cost or boundary
Accumulation	Larger effective batch with fewer resident activations	Latency and normalization/mode subtleties
Momentum/AdamW	History-aware/adaptive updates	State, memory, recovery obligations
Frequent checkpoints	Less lost work	Bandwidth, cost, consistency
Deterministic execution	Easier diagnosis	Possible slowdown, no portability guarantee
Repeated validation	Selection/stopping guidance	Development overfitting

Choose the simplest setup that can falsify the hypothesis. First establish that implementation matches mathematics. Larger models, distributed training, and dashboards cannot compensate for failure at that boundary. Optimize quality and cost together rather than maximizing infrastructure complexity.

K - Difficult interview question and assessment bank

M1-P1 is the difficult interview question, counted once in this exact 28-prompt bank. No answers are supplied. Explain assumptions and discriminating evidence.

Beginner prompts - 5

1. M1-B1: What is learned in $\hat{y} = wx + b$, and what remains fixed during one step?
2. M1-B2: How do a batch, a step, and an epoch differ?
3. M1-B3: What does a negative parameter gradient mean locally?
4. M1-B4: Why is the loss usually reduced to a scalar before calling backward?
5. M1-B5: Why is lower training loss not proof of better production predictions?

Intermediate prompts - 5

1. M1-I1: Trace the shapes through `nn.Linear(1,1)` and mean squared error.
2. M1-I2: What happens after two fresh backwards without clearing gradients?
3. M1-I3: How do `eval()` and `no_grad()` differ, and when are both needed?
4. M1-I4: What distinguishes a leaf parameter from a non-leaf prediction tensor?
5. M1-I5: Why must input normalization be fitted on training data only?

Senior prompts - 5

1. M1-S1: When can a zero gradient lead to a different update from `None`?
2. M1-S2: Which assumptions make a sampled mean gradient unbiased?
3. M1-S3: How would you normalize accumulation across unequal valid-token counts?
4. M1-S4: Why does restoring only weights change momentum or Adam continuation?
5. M1-S5: How would you determine whether a memory failure comes from activations or optimizer state?

Principal prompts - 5

1. M1-P1: A resumed job has identical weights and next-batch loss but takes a different next update; what evidence is needed to establish the cause and decide whether the run remains comparable?
2. M1-P2: Which guarantees would you require before calling a training artifact reproducible?
3. M1-P3: How would you distinguish an implementation regression from a distribution shift?
4. M1-P4: When should improved offline metrics still fail a deployment gate?
5. M1-P5: What assumptions would you challenge in a 16-bytes-per-parameter capacity estimate?

Design prompts - 3

1. M1-D1: Design a versioned contract connecting training preprocessing and inference.
2. M1-D2: Design a minimal CPU experiment that separates optimization correctness from generalization evidence.
3. M1-D3: Design a checkpoint boundary that makes recovery testable without storing a computation graph.

Troubleshooting prompts - 3

1. M1-T1: Predictions are `[32,1]` and targets are `[32]`; how would you investigate unexpectedly smooth training?
2. M1-T2: Parameters stop changing after a refactor although gradients remain nonzero; what would you inspect?
3. M1-T3: A mixed-precision run skips many updates; how would you isolate numerical and data causes?

Whiteboard prompts - 2

1. M1-W1: Derive both gradients and one SGD update for an arbitrary two-example scalar regression batch.
2. M1-W2: Derive the correctly weighted mean loss for two microbatches with different numbers of valid targets.

L - Independent CPU PyTorch lab

1 Objective

Verify the specified step, distinguish backward/update/clearing, train with separate partitions, and compare uninterrupted versus restored next updates. This is the full Lab 1. The independent PyTorch runtime has not been executed in this research.

2 Architecture

A Python process owns CPU `nn.Linear` models, numeric tensors, SGD, and an in-memory checkpoint. The arithmetic probe has no momentum. The separate training exercise uses momentum 0.9 so restoration requires nonempty optimizer state. Six training, three validation, and three test points have disjoint inputs, all following the deliberately trivial $y = 2x$ relation.

3 Prerequisites

Use Python 3.11+ supported by your chosen PyTorch wheel, basic Python functions, and the arithmetic above. No CUDA, account, dataset, or application dependency is required. These PowerShell commands are future learner instructions, not commands executed during synthesis. From the workspace root, create a separate environment under research, not the application's environment:

```
$lab = Join-Path $PWD '.copilot-tracking/research/labs/module-1'
New-Item -ItemType Directory -Force -Path $lab | Out-Null
$env:UV_CACHE_DIR = Join-Path $lab 'uv-cache'
uv venv (Join-Path $lab '.venv') --python 3.11
$python = Join-Path $lab '.venv/Scripts/python.exe'
uv pip install --python $python torch --index-url https://download.pytorch.org/whl/cpu
uv pip install --python $python ipykernel ipywidgets ruff tqdm pytest
```

Use the official installer selector for a compatible CPU wheel (S09). Extra tools follow learning-environment conventions; only PyTorch is a third-party runtime requirement of this script. Pin tested versions before reproducible distribution. Save the code as .copilot-tracking/research/labs/module-1/train_step.py, then run:

```
& $python -B (Join-Path $lab 'train_step.py')
```

4 Code

```
#!/usr/bin/env python3
# Copyright (c) Microsoft Corporation.
# SPDX-License-Identifier: MIT
"""Inspect a scalar training step and a CPU checkpoint round trip.

Run with Python -B in a separate PyTorch environment. Writes no files.
"""

from __future__ import annotations

import copy
import io
import logging
import random
import sys
from typing import Any

import torch
from torch import nn

LOGGER = logging.getLogger(__name__)
DTYPE = torch.float64

def make_model() -> nn.Linear:
    """Create CPU leaf parameters with fixed initial values."""
    model = nn.Linear(1, 1, dtype=DTYPE, device="cpu")
    with torch.no_grad():
        model.weight.fill_(0.5)
        model.bias.fill_(0.0)
    return model

def objective(model: nn.Linear, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
    """Compute scalar MSE after validating the tensor contract."""
    prediction = model(x)
```

```

if prediction.shape != y.shape or x.ndim != 2 or x.shape[1] != 1:
    raise ValueError("Expected inputs, predictions, targets with shape [B, 1]")
loss = (prediction - y).square().mean()
if not bool(torch.isfinite(loss)):
    raise FloatingPointError("Loss is nonfinite; inspect data and predictions")
return loss

def score(model: nn.Linear, x: torch.Tensor, y: torch.Tensor) -> float:
    """Evaluate without building a graph; caller restores training mode."""
    model.eval()
    with torch.no_grad():
        return objective(model, x, y).item()

def update(
    model: nn.Linear, optimizer: torch.optim.Optimizer,
    x: torch.Tensor, y: torch.Tensor,
) -> float:
    """Apply one update and return the pre-update loss."""
    model.train()
    optimizer.zero_grad(set_to_none=True)
    loss = objective(model, x, y)
    loss.backward()
    for parameter in model.parameters():
        if parameter.grad is None or not bool(torch.isfinite(parameter.grad).all()):
            raise FloatingPointError("Expected a finite gradient for each parameter")
    optimizer.step()
    return loss.item()

def verify_single_step() -> None:
    """Assert forward, backward, mutation, and clearing independently."""
    x = torch.tensor([[1.0], [2.0]], dtype=DTYPE)
    y = torch.tensor([[2.0], [4.0]], dtype=DTYPE)
    model = make_model()
    optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
    optimizer.zero_grad(set_to_none=True)
    assert model.weight.is_leaf and model.weight.grad_fn is None
    torch.testing.assert_close(model(x), torch.tensor([[0.5], [1.0]], dtype=DTYPE))
    loss = objective(model, x, y)
    assert loss.grad_fn is not None and loss.ndim == 0
    torch.testing.assert_close(loss, torch.tensor(5.625, dtype=DTYPE))
    loss.backward()
    assert model.weight.grad is not None and model.bias.grad is not None
    torch.testing.assert_close(model.weight.grad, torch.tensor([[ -7.5]], dtype=DTYPE))
    torch.testing.assert_close(model.bias.grad, torch.tensor([ -4.5], dtype=DTYPE))
    assert model.weight.item() == 0.5 and model.bias.item() == 0.0
    old_gradient = model.weight.grad.clone()
    optimizer.step()
    torch.testing.assert_close(model.weight, torch.tensor([[1.25]], dtype=DTYPE))
    torch.testing.assert_close(model.bias, torch.tensor([0.45], dtype=DTYPE))
    torch.testing.assert_close(model.weight.grad, old_gradient)
    after = score(model, x, y)
    torch.testing.assert_close(torch.tensor(after, dtype=DTYPE),
                               torch.tensor(0.59625, dtype=DTYPE))
    LOGGER.info("step loss=%.6f dw=-7.500000 db=-4.500000", loss.item())
    LOGGER.info("new w=%.6f b=%.6f same_batch_mse=%.6f",
                model.weight.item(), model.bias.item(), after)
    optimizer.zero_grad(set_to_none=True)
    assert all(p.grad is None for p in model.parameters())
    # Fresh graphs, unchanged weights: demonstrate accumulation deliberately.
    objective(model, x, y).backward()
    first = model.weight.grad.clone()
    objective(model, x, y).backward()
    torch.testing.assert_close(model.weight.grad, 2 * first)

```

```

optimizer.zero_grad(set_to_none=True)

def make_data() -> tuple[dict[str, tuple[torch.Tensor, torch.Tensor]], dict[str, Any]]:
    """Fit preprocessing only on fixed training records.

    Returns:
        Split tensors and serializable preprocessing metadata.
    """
    raw = {
        "train": torch.tensor([[-3.0], [-2.0], [-1.0], [1.0], [2.0], [3.0]], dtype=DTYPE),
        "validation": torch.tensor([[-2.5], [0.5], [2.5]], dtype=DTYPE),
        "test": torch.tensor([[-1.5], [0.0], [1.5]], dtype=DTYPE),
    }
    groups = [set(values.flatten().tolist()) for values in raw.values()]
    assert all(groups[i].isdisjoint(groups[j])
               for i in range(3) for j in range(i + 1, 3))
    mean = raw["train"].mean().item()
    scale = raw["train"].std(correction=0).item()
    if scale <= 0:
        raise ValueError("Training feature must have positive scale")
    preprocessing = {"kind": "standardize", "mean": mean, "scale": scale,
                    "fit_split": "train", "feature_order": ["x"]}
    splits = {name: ((values - mean) / scale, 2 * values)
              for name, values in raw.items()}
    return splits, preprocessing

def snapshot(
    model: nn.Linear, optimizer: torch.optim.Optimizer,
    preprocessing: dict[str, Any], config: dict[str, Any],
) -> bytes:
    """Serialize a trusted epoch-boundary checkpoint to memory.

    Any is confined to metadata and the library serialization boundary.
    """
    payload = {
        "schema_version": 1, "architecture": "Linear(1,1)",
        "model": model.state_dict(), "optimizer": optimizer.state_dict(),
        "torch_rng": torch.get_rng_state(), "python_rng": random.getstate(),
        "config": config, "preprocessing": preprocessing,
        "input_schema": {"features": ["x"], "shape": [None, 1], "dtype": "float64"},
        "progress": {"epoch_completed": 1, "step": 3, "next_batch_cursor": 0},
        "dataset": "fixed-disjoint-scalar-v1", "code_revision": "module-1-v1",
        "runtime": {"torch": str(torch.__version__), "python": sys.version},
        "scheduler": None, "scaler": None,
    }
    with io.BytesIO() as buffer:
        torch.save(payload, buffer)
        return buffer.getvalue()

def restore(blob: bytes) -> tuple[nn.Linear, torch.optim.SGD, dict[str, Any]]:
    """Restore only this exercise's trusted, compatible checkpoint."""
    with io.BytesIO(blob) as buffer:
        payload = torch.load(buffer, map_location="cpu", weights_only=True)
    if payload["schema_version"] != 1 or payload["architecture"] != "Linear(1,1)":
        raise ValueError("Unsupported checkpoint schema or architecture")
    config = payload["config"]
    model = make_model()
    optimizer = torch.optim.SGD(model.parameters(), lr=config["lr"],
                                momentum=config["momentum"])
    model.load_state_dict(payload["model"])
    optimizer.load_state_dict(payload["optimizer"])
    # Restore RNG after construction, which itself consumes random numbers.
    torch.set_rng_state(payload["torch_rng"])

```

```

random.setstate(payload["python_rng"])
model.train()
return model, optimizer, payload

def verify_resume(
    model: nn.Linear, optimizer: torch.optim.SGD, blob: bytes,
    x: torch.Tensor, y: torch.Tensor,
) -> None:
    """Compare uninterrupted and restored next updates, including momentum."""
    ids_a = torch.randperm(len(x))[:2]
    loss_a = update(model, optimizer, x[ids_a], y[ids_a])
    state_a = torch.get_rng_state().clone()
    resumed, resumed_optimizer, _ = restore(blob)
    ids_b = torch.randperm(len(x))[:2]
    loss_b = update(resumed, resumed_optimizer, x[ids_b], y[ids_b])
    assert torch.equal(ids_a, ids_b) and loss_a == loss_b
    assert torch.equal(state_a, torch.get_rng_state())
    for original, recovered in zip(model.parameters(), resumed.parameters(), strict=True):
        torch.testing.assert_close(original, recovered, rtol=0, atol=0)
        torch.testing.assert_close(
            optimizer.state[original]["momentum_buffer"],
            resumed_optimizer.state[recovered]["momentum_buffer"], rtol=0, atol=0,
        )
    LOGGER.info("next-step equivalence: PASS (batch, loss, weights, momentum, torch RNG)")

def run_training() -> None:
    """Train, select by validation, and access test only after selection."""
    config = {"seed": 7, "lr": 0.05, "momentum": 0.9,
              "batch_size": 2, "epochs": 80, "dtype": "float64", "device": "cpu"}
    random.seed(config["seed"])
    torch.manual_seed(config["seed"])
    torch.set_num_threads(1)
    torch.use_deterministic_algorithms(True)
    splits, preprocessing = make_data()
    train_x, train_y = splits["train"]
    model = make_model()
    optimizer = torch.optim.SGD(model.parameters(), lr=config["lr"],
                                momentum=config["momentum"])
    best_value = float("inf")
    best_state = copy.deepcopy(model.state_dict())
    step = 0
    for epoch in range(config["epochs"]):
        order = torch.randperm(len(train_x))
        for ids in order.split(config["batch_size"]):
            update(model, optimizer, train_x[ids], train_y[ids])
            step += 1
    if epoch == 0:
        assert step == 3
        blob = snapshot(model, optimizer, preprocessing, config)
        verify_resume(model, optimizer, blob, train_x, train_y)
        # Discard probe updates; resume from the same clean epoch boundary.
        model, optimizer, restored = restore(blob)
        assert restored["preprocessing"] == preprocessing
        assert restored["progress"]["step"] == step
    validation = score(model, *splits["validation"])
    if validation < best_value:
        best_value = validation
        best_state = copy.deepcopy(model.state_dict())
    model.load_state_dict(best_state)
    test_value = score(model, *splits["test"])
    raw_slope = model.weight.item() / preprocessing["scale"]
    raw_intercept = model.bias.item() - raw_slope * preprocessing["mean"]
    LOGGER.info("steps=%d best_validation_mse=%.8g held_out_test_mse=%.8g",
                step, best_value, test_value)

```

```

LOGGER.info("raw-space slope=%.8g intercept=%.8g", raw_slope, raw_intercept)

def main() -> int:
    """Run demonstrations and report failures without hidden side effects."""
    logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
    try:
        verify_single_step()
        run_training()
    except KeyboardInterrupt:
        return 130
    except (AssertionError, ValueError, RuntimeError, FloatingPointError):
        LOGGER.exception("Lab failed; compare the first failing contract with the lesson")
        return 1
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

5 Explanation

`verify_single_step` checks graph/numbers, unchanged weights after backward, movement after step, retained gradients after step, and fresh-graph accumulation. Assertions are teaching checks; do not run with -O. `make_data` checks disjointness and fits normalization once. These tiny noise-free partitions are not evidence of real-population performance even though held-out values are unused for gradients.

`snapshot` serializes after the first epoch's three updates, before the next shuffle. No partial permutation/accumulation exists. `verify_resume` advances one branch, constructs a replacement, restores RNG after construction, and compares the next update exactly. The caller restores the boundary so probes do not alter training. Only torch RNG drives sampling; Python RNG is saved as a precaution. No NumPy RNGs, workers, GPU states, schedulers, or scalars are used.

Validation selects deep-copied inference weights. Pairing those weights with the final optimizer would not be a valid recovery checkpoint: states could be from different steps. Recovery and best-model snapshots have different purposes.

6 Expected output

These are derived expectations, not a captured PyTorch transcript:

```

INFO: step loss=5.625000 dw=-7.500000 db=-4.500000
INFO: new w=1.250000 b=0.450000 same_batch_mse=0.596250
INFO: next-step equivalence: PASS (batch, loss, weights, momentum, torch RNG)

```

Final logging should report 240 training updates, finite validation/test MSE, and raw slope/intercept approaching 2/0. Exact final metrics are not asserted or fabricated. Seeds and deterministic settings do not guarantee cross-release, device, or platform equality (S07). Exact replay is scoped to the same CPU process and compatible runtime. Arithmetic assertions are known; runtime success is pending.

7 Observation

Pause after each operation and predict `.grad`: `None`, a parameter-shaped tensor, or an accumulated value. Verify validation has no graph and never enters `update`. Observe momentum state becoming nonempty. Explain why restoring RNG before replacement-model construction would break the batch-order check.

8 Failures to explore

In a disposable copy, remove clearing, reshape a target to `[B]`, or omit optimizer restoration, one change at a time. Predict the failing contract, execute when the environment is ready, then restore baseline. A passing same-batch check does not establish split validity or readiness. Fix imports in the separate environment, not by modifying application dependencies.

9 Production equivalent

Replace fixed tensors with a versioned reader/split manifest, memory serialization with atomic integrity-checked storage, and example identities with exact dependency, data, and code identities. Add slices and release criteria. Extend continuation state only for features used. Measure a new-process restart before claiming crash recovery. These are mapping exercises, not solved later-module architectures.

Five-question quiz - no answers

1. M1-Q1: Which exact state changes during backward, and which changes during step?
2. M1-Q2: Why must both parameter gradients be computed before either parameter is updated in this example?
3. M1-Q3: What new predictions justify the post-update MSE?
4. M1-Q4: Which dataset is allowed to fit preprocessing, select a snapshot, and supply final test evidence?
5. M1-Q5: What additional state beyond weights is needed for the lab's next-step equivalence check?

Progressive glossary - 10 terms

Term	Meaning in this lesson
Parameter	Learned tensor, such as slope or bias
Tensor	Typed multidimensional array with shape and device
Mini-batch	Subset processed together to estimate an objective or gradient
Forward pass	Evaluation of the parameterized function on inputs
Loss	Numerical objective being minimized
Computation graph	Operation dependencies needed for differentiation
Backpropagation	Reverse application of chain rule through dependencies
Gradient	Local derivatives of loss with respect to parameters

Term	Meaning in this lesson
Optimizer	Rule and optional state converting gradients into updates
Checkpoint	Serialized snapshot sufficient for a stated restoration purpose

Only these ten terms enter the active glossary now. Later lessons extend it after use in a practical exercise, rather than assigning a large vocabulary to memorize.

Progress log and evidence of learning

Capability	Covered material	Earned evidence required	Current status
Explain lifecycle	Journey and production boundaries	Identify loops and gates	Not yet assessed
Derive a step	Arithmetic and chain rule	Reproduce without looking	Not yet assessed
Implement update	Tensor/mutation contracts	Run and explain assertions	Not yet assessed
Separate data uses	Train-only transform and held-out splits	Explain controls and limitations	Not yet assessed
Resume state	Momentum/RNG serialization	Same next batch and update	Not yet assessed
Estimate resources	State/activation/overhead ledger	Defend assumptions, then measure	Not yet assessed

Initial log, 2026-09-13: plan and first lesson available; no learner answers, quiz results, lab transcript, or grades submitted. Module 1 is ready for study, not passed. Modules 2-28 are planned and not yet assessed. Record future session date, module/prompt ID, evidence location, score/10, remaining weakness, remediation, and next gate here. Reading is exposure, not mastery. Advanced CNN/RNN internals, transformer training, distributed execution, real-world evaluation, and production deployment are neither completed nor assessed by this first lesson.

Curriculum briefings and cumulative laboratory roadmap

The following briefings define future learning, not completed teaching. Every module has a bounded exercise feeding its level lab. Exercise durations are included in module hours; the cumulative labs reuse those outputs. Labs 2-10 preserve objectives, architecture, prerequisites, exact API sketches, observations, expected outputs, faults, production equivalents, and gates. Their full nine-field executable lesson expansion is deferred until the corresponding live module and version selection.

Level 1 foundations

Module 1 Numerical learning and experimental reasoning

Treat learning as an explicit numerical process rather than an API invocation. Connect vectors, matrix multiplication, scalar loss, and chain rule to a parameter update. Separate training examples from validation evidence and connect probability estimates to decisions with asymmetric consequences. The aim is vocabulary for questioning experiments, not introductory programming. Explain why lower training loss does not prove generalization, why gradients are not predictions, and why seeds do not guarantee cross-hardware reproducibility. The developed lesson above uses $x = [1, 2]$, $y = [2, 4]$, $w = 0.5$, and $b = 0$ as the single canonical example.

- Prerequisites: Algebra, basic functions, reading short numerical code
- Suggested hours: 10
- Deliverable: Annotated forward-loss-backward-update trace and split rationale
- Bounded exercise: 90 minutes to derive both gradients, predict each API's mutation, and compare the full Lab 1 assertions; retain a separate execution-pending label if PyTorch is unavailable
- Level lab connection: Lab 1 below, implemented in Module 1 section L
- Gate: Hand and automatic gradients agree within 1e-5; explain three ways lower training loss can mislead

Lab 1 One auditable update and independent recovery probe

Use the full nine-field Module 1 section L lab above, not a second scalar example. Its arithmetic expectations are loss 5.625, gradients -7.5/-4.5, updated parameters 1.25/0.45, and new MSE 0.59625. Its separate held-out training exercise checks in-memory momentum/RNG continuation. Runtime, convergence, and recovery assertions remain unexecuted. This connection deliberately has one authoritative code listing.

Level 2 deep learning

Module 2 Neural networks and representation learning

A neural network learns intermediate representations that make a prediction task easier, rather than relying entirely on hand-designed features. Compare a linear baseline with a small multilayer perceptron and explain the role of nonlinear activations, initialization, and residual paths. Relate cross-entropy to classification and squared error to regression without assuming either metric is the business objective. Use convolutional and recurrent models as contrasting inductive biases, not as a detour into every architecture family. Track shapes through layers and identify which parameters share information across positions. Regularization, dropout, normalization, and early stopping should become testable choices. A larger network may fit a noisy dataset better while providing worse decisions; document that distinction through train-validation curves and error slices rather than an architectural popularity contest.

- Prerequisites: Module 1
- Suggested hours: 14
- Deliverable: Linear versus MLP comparison with shape diagram and error slices
- Bounded exercise: In 90 minutes, count parameters in a two-layer MLP, draw shapes, and compare one fixed-budget linear/MLP pair; introduce one overfitting condition
- Level lab connection: Lab 2 model and dataset baseline

- Gate: Explain parameter counts exactly and attribute an overfitting curve to evidence rather than guessing

Module 3 Optimization and training schedules

SGD follows sampled gradients; momentum smooths their history; Adam estimates first and second moments; AdamW decouples weight decay from those moments. Work through why these choices change update magnitude and state storage. Learning rate is a policy over updates, not a universal property of a model. Compare warmup, constant rate, and decay under a fixed token or example budget. Distinguish an epoch, dataloader iteration, microbatch, optimizer step, and scheduler step. Gradient accumulation changes the effective batch without making all examples resident at once, but it is not automatically equivalent when normalization, dropout, clipping, or loss weighting differs. Reason about noise and convergence before increasing the batch to fill hardware. Interpret gradient norms and learning curves together; elapsed epochs alone conceal differences in actual work.

- Prerequisites: Module 2
- Suggested hours: 18
- Deliverable: Controlled SGD, Adam, and AdamW runs with an update-count ledger
- Bounded exercise: In two hours, compare one FP32 batch of 64 with four of 16, write the denominator/update ledger, then compare three optimizer trajectories under one budget
- Level lab connection: Lab 2 accumulation and optimizer sweep
- Gate: Calculate effective batch/updates unambiguously; reproduce controlled accumulated versus large-batch gradients within $1e-5$

Module 4 Precision, memory, and reproducible training

Precision is a set of choices for weights, activations, gradients, optimizer states, and individual operations. Automatic mixed precision selects operation dtypes; it does not mean every tensor becomes half precision or that a master weight copy always exists. FP16 gradient scaling addresses underflow, while BF16's different exponent range changes the tradeoff. Inspect actual tensor dtypes and optimizer state after the first step. Distinguish allocated tensors, allocator reservations, device context, temporary workspaces, and checkpoint storage. Activation checkpointing trades recomputation for retained activations and can change runtime behavior. Reproducibility requires data order, transforms, random state, scheduler, and optimizer state as well as weights. Learn to explain a resume discrepancy before promising bitwise replay across software or hardware changes. Official AMP examples support the accumulation and clipping order.

- Prerequisites: Module 3
- Suggested hours: 16
- Deliverable: Precision/memory ledger and interrupted/resumed training comparison
- Bounded exercise: In two hours, inventory dtypes before/after first step, resume at one clean boundary, and compare a deliberately incomplete checkpoint; GPU AMP branch only when supported
- Level lab connection: Lab 2 state, precision, and restart tracks
- Gate: Account for persistent tensors; diagnose one nonfinite-gradient case and one incorrect resume

Lab 2 Training under a fixed work budget

Objective: Separate optimization effects from systems effects. Architecture: a synthetic two-class dataset feeds a linear baseline and two-layer MLP; metrics record updates, loss, elapsed time, and memory.

Prerequisites: modules 1-4, CPU PyTorch, optional supported GPU for AMP. Use 1,024 examples, a fixed train-validation split, and no dropout/BatchNorm for accumulation equivalence.

API surface: `torch.nn.Linear`, `torch.nn.ReLU`, `torch.nn.functional.cross_entropy`, `torch.optim.SGD`, `torch.optim.Adam`, `torch.optim.AdamW`, `torch.autocast`, `torch.amp.GradScaler`, and `torch.nn.utils.clip_grad_norm_`. Inspect `Optimizer.state_dict()`; use `torch.cuda.max_memory_allocated()` only on the GPU track (C01-C03).

```
For each effective batch of 64 examples:
  clear gradients once
  process four equal microbatches of 16
  backpropagate each mean loss divided by four
  if FP16 scaling is enabled, unscale once after all four backwards
  clip once, apply one optimizer update, update scaler once
  advance the update-based scheduler only after a successful update
  record updates, examples seen, rate, gradient norm, and loss
```

Compare one batch of 64 with four of 16 in FP32. Sweep three learning rates for each optimizer at the same examples-seen budget. Interrupt at an effective-batch boundary, persist complete state, and resume.

Expected output, unexecuted: matched controlled gradients, distinct curves, state inventory, resume comparison. No optimizer is expected to win universally.

Observe underflow, skipped FP16 updates, wrong divisors, clipping before unscale, and missing scheduler state. Introduce an incomplete final accumulation group to explain why dividing by four can be wrong. Production equivalent: diagnose a training regression before fleet changes. Pass when equivalence meets tolerance and all introduced faults are identified.

Level 3 LLM internals

Module 5 Tokenization, embeddings, and language objectives

An LLM predicts over token IDs, not words or raw business concepts. Examine how byte-pair or related subword tokenization changes sequence length for English, code, identifiers, and multilingual text.

Embedding lookup converts IDs into learned vectors; it is not the same operation as producing a sentence embedding for retrieval. Connect next-token cross-entropy to a shifted target sequence and explain the roles of padding masks and loss masks. Chat formatting is part of the model contract: role delimiters, beginning/end tokens, and generation prompts can alter behavior even when visible text looks identical.

Compare a base model with an instruction-tuned model without changing the tokenizer accidentally.

Vocabulary size affects embedding and output-projection storage; tokenization also changes billing, context usage, throughput, and evaluation comparability.

- Prerequisites: Modules 2-4
- Suggested hours: 14
- Deliverable: Token/label inspection for 30 domain examples and two templates
- Bounded exercise: In 90 minutes, tokenize 30 permitted strings, trace five shifted-label examples, and compare two templates for duplicate tokens and masks

- Level lab connection: Lab 3 input and objective contracts
- Gate: Detect seeded duplicate tokens/off-by-one labels; explain why cross-tokenizer perplexities are not directly comparable

Module 6 Transformer tensor mechanics

Trace a decoder block from batch-by-sequence-by-hidden input to vocabulary logits. Queries, keys, and values are learned projections, with separate head counts under grouped-query attention. Scaled dot products, causal masking, softmax, and value aggregation mix earlier positions; the feed-forward network transforms each position's features. Residual connections and normalization control information and gradient flow. RoPE rotates query/key components by position rather than appending a position label to the output. Contrast RMSNorm with LayerNorm and a gated FFN with a basic two-layer MLP. Calculate intermediate shapes before inspecting tensors. Mathematical attention scores have quadratic sequence dimensions, but optimized attention can avoid materializing that entire matrix. Do not confuse a pedagogical score tensor with the memory layout of a production attention kernel or claim RoPE alone guarantees long-context quality.

- Prerequisites: Module 5 and Module 1 matrix multiplication
- Suggested hours: 20
- Deliverable: Annotated GQA decoder block with parameter/activation shapes
- Bounded exercise: In two hours, derive the Lab 3 Q/K/V and FFN shapes, inspect hooks, then alter future tokens in one causal-invariance fixture
- Level lab connection: Lab 3 reference attention and shape ledger
- Gate: Derive all shapes for $B=2$, $T=16$, $D=256$, $H_q=8$, $H_{kv}=2$, $head_dim=32$; pass causal invariance

Module 7 Autoregressive generation and context

Generation repeatedly evaluates a conditional distribution, chooses a token, and extends context. Distinguish raw logits from probabilities and compare greedy selection, temperature, top-k, and top-p sampling. Stopping rules and maximum output length affect both user experience and evaluation results. KV caching preserves past keys and values to avoid recomputing them, but it adds memory proportional to retained context and active sequences. Prefill processes the prompt; decode extends it incrementally. Long-context support is therefore both a numerical/model capability and a scheduling constraint. Test retrieval from different positions, not only whether a long input is accepted. Compare cached and uncached generation with matched settings and tolerances. Explain why deterministic sampling settings do not eliminate all hardware-dependent numerical differences, and why streaming is transport behavior rather than a different learned objective.

- Prerequisites: Module 6
- Suggested hours: 14
- Deliverable: Logit/token/cache/stopping generation trace
- Bounded exercise: In 90 minutes, decode eight tokens with and without cache, compare logits under a stated tolerance, and vary one stopping/sampling rule
- Level lab connection: Lab 3 cached continuation
- Gate: Reconcile cache/non-cache outputs; separate first-token from inter-token latency

Lab 3 A decoder you can explain

Objective: Expose tensors and cache without a large download. Architecture: synthetic IDs enter a tiny random Llama-style causal model; hooks capture shapes and a reference path computes causal attention. Prerequisites: modules 5-7 and CPU PyTorch/Transformers in a dedicated environment. Random weights suffice for mechanics, not language-quality claims.

APIs: `transformers.LlamaConfig(vocab_size=128, hidden_size=256, intermediate_size=512, num_hidden_layers=2, num_attention_heads=8, num_key_value_heads=2, max_position_embeddings=256)`, `transformers.LlamaForCausalLM(config)`, `Module.register_forward_hook()`, `model(input_ids=ids, labels=ids, use_cache=False)`, `model(input_ids=prefix, use_cache=True)`, and returned `past_key_values` in the next call. Supply masks/positions for combined cache/new tokens per pinned API (C04-C05).

At B=2, T=16: hidden (2,16,256); query projections (2,16,256); K/V projections (2,16,64); reshaped queries (2,8,16,32); unexpanded K/V (2,2,16,32). Four query heads share one KV head. Conceptual scores are (2,8,16,16), logits (2,16,128), and FFN gate/up intermediate dimension 512.

Expected output, unexecuted: shape ledger, finite loss, cached/full-prefix logits matching within chosen FP32 tolerance. Alter future tokens and test earlier logits in eval mode. Observe wrong mask polarity, cache positions, and twice-shifted labels. Looking ahead lowers loss through leakage. Production equivalent: conversion, cache-backend, or context regression diagnosis. Pass the shape gate and retain failing/repaired traces.

Level 4 LLM training and distributed GPUs

Module 8 Pretraining data and compute strategy

Pretraining from random initialization learns broad token statistics; continued pretraining adapts an existing model to another distribution, often without instruction labels. Both require data rights, provenance, filtering, deduplication, and independent evaluation. Design a token mixture across domains and languages, then distinguish mixture weights from an ordered learning curriculum. Synthetic data can add targeted coverage but can also amplify teacher errors, duplicated reasoning, and benchmark contamination. Measure diversity and verification cost, not only generated token count. Use scaling-law research to form hypotheses about model size, tokens, and compute, while recognizing that compute-optimal pretraining is not necessarily lifecycle-cost-optimal serving. Continued training can erase useful capabilities. Compare a domain-only mixture with replay of general data, and budget checkpointing, retries, preprocessing, and evaluation alongside accelerator time rather than treating FLOPs as the complete bill.

- Prerequisites: Modules 1-7
- Suggested hours: 20
- Deliverable: Versioned corpus manifest, mixture experiment, compute budget
- Bounded exercise: In two hours, create two synthetic domain manifests, split by family before packing, calculate a 100-update token budget, and compare continuation mixtures
- Level lab connection: Lab 4 data/pretraining/continuation track
- Gate: Trace each shard to a permitted source; distinguish duplicate-free splits from unproven absence of pretraining contamination

Module 9 Distributed training and GPU fabrics

Map each parallel technique to the tensors it partitions and messages it creates. Data parallelism processes different examples; FSDP or ZeRO shards training state. Tensor parallelism splits operations, pipeline parallelism splits layers, sequence parallelism can shard normalization/residual activations, and context parallelism partitions long-sequence attention work. Expert parallelism routes tokens to MoE experts, so load balance and all-to-all traffic matter even when active parameter count is small. HBM capacity, bandwidth, FLOPs, PCIe, NVLink, and inter-node InfiniBand or RoCE constrain different phases. NCCL collectives require matching participation and compatible shapes across ranks. A larger fleet can reduce useful throughput when communication dominates. Sketch the device mesh, place frequent communication on faster links, and measure scaling efficiency before recommending another parallel dimension.

- Prerequisites: Modules 4, 6, 8 and network architecture knowledge
- Suggested hours: 20
- Deliverable: DP/TP/PP/SP/CP/EP map and two-rank trace or explicit hardware gap
- Bounded exercise: In two hours, map six partition strategies, calculate ring traffic, and run the two-rank sum only on approved hardware; otherwise analyze a labeled replay
- Level lab connection: Lab 4 optional distributed branch, reinforced by Labs 7/9
- Gate: Explain all-reduce/all-gather/reduce-scatter/all-to-all; identify topology and rank-participation failures

Module 10 Post-training objectives and reward learning

Supervised fine-tuning teaches behavior from demonstrations; preference learning uses comparisons rather than assuming one reference answer is uniquely correct. Traditional RLHF commonly combines SFT, a learned reward model, and policy optimization with a reference-policy constraint. DPO optimizes preference pairs directly and does not require a separately trained reward model in its standard form. GRPO generates groups of completions and derives relative advantage from rewards; it moves cost into rollout generation and reliable reward computation. Reward can be learned, human-provided, or mechanically verifiable, and each has blind spots. Study reward hacking, length bias, zero-variance groups, and policy collapse. Keep reward improvement separate from held-out task improvement. Implementation defaults evolve, so explicitly choose normalization, reference behavior, and loss variant rather than treating every GRPO implementation as the original paper's algorithm.

- Prerequisites: Modules 3, 7, 8
- Suggested hours: 16
- Deliverable: SFT/RLHF/DPO/GRPO decision record and audited reward/preference data
- Bounded exercise: In 90 minutes, audit ten preference pairs, score two groups of four arithmetic answers including an all-equal group, and record reward blind spots
- Level lab connection: Lab 4 post-training worksheet; objective training optional
- Gate: Identify objects requiring gradients in each method; reject a reward-hacked candidate using independent evidence

Lab 4 A miniature training program with a restart

Objective: Exercise training at toy scale without inferring frontier behavior. Architecture: two permitted synthetic text domains, fixed tokenizer, tiny causal model, checkpoint store, held-out evaluator, and separate reward worksheet. Prerequisites: level 3 model, Module 8 manifest, optional two Linux GPUs.

APIs: `datasets.Dataset.from_dict`, `transformers.AutoModelForCausalLM.from_config`, `torch.optim.AdamW`, `model.save_pretrained`, `optimizer/scheduler state_dict`, `torch.distributed.init_process_group`, `torch.distributed.all_reduce`, `torch.nn.parallel.DistributedDataParallel`. Each GPU process selects its device. Use NCCL only on supported GPU track; CPU backend simulation is not an NCCL benchmark (C06-C10).

Run 100 bounded updates from random initialization, then 50 continued updates on domain B. Compare domain-only with a mixture retaining A. Split by family before packing. Save model, optimizer, scheduler, RNG, sampler/cursor, and count at update 75; compare resume with uninterrupted execution. Two-rank branch: summing rank values 1 and 2 must return 3 on both ranks.

Create ten prompt/chosen/rejected pairs for `trl.DPOTrainer`. Separately score groups of four completions with a deterministic arithmetic checker compatible with `trl.GRPOTrainer(reward_funcs=...)`. Compute advantages including all-equal rewards. Training these variants is optional; data/memory contract analysis is required.

Expected output, unexecuted: corpus lineage, both-domain loss slices, declared resume-tolerance comparison, reward failure table. Observe forgetting, duplicates, missing sampler state, and missing collective participation. Production equivalent: restartable training with credible data/reward governance. Pass when no evaluated record is in the toy training split and all resume discrepancies are explained.

Level 5 fine-tuning

Module 11 Full fine-tuning and LoRA

Full fine-tuning updates the selected base parameters and typically creates gradients and optimizer state for a large fraction of the model. LoRA freezes base weights and learns a low-rank update, usually represented as two smaller matrices. Derive the extra parameter count before selecting rank and target modules. The saving is primarily trainable state; forward computation and activation storage do not disappear. Compare attention-only targets with wider linear-layer coverage under a matched task and work budget. Distinguish the training objective, such as SFT or DPO, from the update mechanism, such as LoRA. They are independent choices. Merging an adapter changes the artifact and may change numerical behavior, especially around quantization. Preserve the exact base revision and tokenizer contract, and evaluate the deployed representation rather than assuming the training checkpoint is operationally interchangeable.

- Prerequisites: Level 4
- Suggested hours: 18
- Deliverable: Full-update/LoRA comparison with trainable counts and peak memory
- Bounded exercise: In two hours, derive target-layer counts at rank 8, hash frozen weights, and compare one toy full-update run with LoRA under a matched budget
- Level lab connection: Lab 5 adapter identity and base-freezing checks
- Gate: Predict counts exactly; show frozen parameters receive no updates

Module 12 QLoRA, quantization, and alternative adapters

QLoRA combines a quantized frozen base with trainable low-rank parameters; it is not full-model integer training. Distinguish storage precision, compute precision, quantization scales, and optimizer precision. Post-training quantization can reduce deployment cost but depends on representative calibration, kernel support,

and acceptable quality change. Compare weight-only INT8/INT4, floating-point low-precision formats, and quantized KV caches without assuming equal support or speed. Alternative parameter-efficient methods modify different surfaces: bottleneck adapters insert modules, prompt tuning learns input embeddings, prefix tuning adds learned attention prefixes, and IA3 rescales activations with learned vectors. Their serving implications differ, particularly context overhead, merging, and multi-adapter batching. Choose based on task evidence and runtime support, not the smallest checkpoint alone. Keep a compatibility matrix tied to the actual model, accelerator, library versions, and export path.

- Prerequisites: Modules 4, 11
- Suggested hours: 16
- Deliverable: QLoRA experiment and full FT/LoRA/QLoRA/adapters/prefix/prompt/IA3 comparison
- Bounded exercise: In two hours, complete a seven-method compatibility/state table, reconcile packed weights with metadata, and run one supported QLoRA conversion or label hardware evidence pending
- Level lab connection: Lab 5 optional QLoRA and deployment-quality checks
- Gate: Reconcile metadata/storage and demonstrate an agreed held-out bound after conversion

Module 13 Adaptation experiment design and transfer

Decide whether the task needs new knowledge, a better interface, a constrained output, or a changed behavior before reaching for training. Retrieval can supply fresh authorized evidence; prompting can clarify instructions; SFT can teach formats; continued pretraining can shift domain statistics. These approaches solve overlapping but different problems. Build a baseline ladder that compares them against the same held-out questions and includes human escalation or no model as legitimate outcomes. Audit synthetic demonstrations for teacher errors, copied answers, and coverage gaps. Track learning curves by data volume and evaluate general-capability retention as well as domain improvement. Test adapter transfer to a new base revision as an explicit compatibility experiment rather than a routine upgrade. Record the cheapest intervention that satisfies the requirement and the evidence that would justify a more expensive one.

- Prerequisites: Modules 8, 10-12
- Suggested hours: 14
- Deliverable: Adaptation decision record with prompting, retrieval, and tuned baselines
- Bounded exercise: In two hours, lock one task slice, compare three intervention contracts, and specify one base-revision transfer/retention test; reject options without evidence
- Level lab connection: Lab 5 baseline ladder and final adaptation decision
- Gate: Locked split/fixed budget; defend selection or rejection without training loss

Lab 5 Adapt a model without losing its identity

Objective: Compare base, prompt baseline, and LoRA on a narrow formatting task. Architecture: approved local small causal model/tokenizer consumes synthetic ticket summaries; runner emits adapter; evaluator tests format and general retention. Prerequisites: level 4, permitted 100M-1B model selected for memory, PEFT/Transformers, 200 training and 100 held-out reviewed items. These counts teach mechanics, not high-confidence production conclusions.

APIs: `peft.LoraConfig(r=8, lora_alpha=16, target_modules=["q_proj", "v_proj"], bias="none", task_type="CAUSAL_LM")`, `peft.get_peft_model`, `PeftModel.save_pretrained`, `PeftModel.from_pretrained`. Check actual module names before applying Llama-style targets. Optional QLoRA uses

```
transformers.BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",  
bnb_4bit_compute_dtype=torch.bfloat16) and peft.prepare_model_for_kbit_training
```

 on supported hardware (C11-C12).

Count targets, hash frozen weights before/after, inspect masks on ten examples. Save adapter plus base revision, tokenizer, template, data digest, evaluation digest. Reload in a fresh process and compare with training-time output. Do not require unsupported merges into a quantized base. Use a smaller toy for full FT if the selected checkpoint exceeds memory.

Expected output, unexecuted: trainable inventory, unchanged base hashes, reload parity, paired quality. Predeclare an educational retention bound, such as at most five percentage points lost; report uncertainty instead of claiming 100 examples prove it. Observe wrong targets, missing adapters, changed tokenizer, unsupported quantization, schema-valid wrong answers. Production equivalent: governed adapter release with independently tested base binding. Pass identity checks and defend the adaptation decision.

Level 6 evaluation

Module 14 Model evaluation and statistical evidence

Model evaluation isolates capabilities under a controlled prompt and generation protocol. Use perplexity for compatible tokenization/objective comparisons and task metrics for the behavior that matters. Benchmark scores are samples, not immutable properties of a checkpoint. Separate data leakage from potential pretraining contamination, and state when the latter cannot be ruled out. Use paired comparisons on the same independent tasks, confidence intervals, effect sizes, and predeclared practical margins. Resample at the task or customer cluster level when repeated turns are dependent. Human evaluation needs a rubric, blind assignment, disagreement handling, and protected reviewer conditions. LLM-as-judge evaluations need calibration against humans and checks for position, verbosity, and self-preference biases. Repeating a biased judge many times does not fix bias. Distinguish inconclusive evidence from evidence of equivalence and avoid tuning repeatedly against the final holdout.

- Prerequisites: Modules 1, 7, 13
- Suggested hours: 18
- Deliverable: Paired model report with intervals, rubric, contamination statement, blind labels
- Bounded exercise: In two hours, align 100 task IDs, compute a paired interval, reverse half the judge presentations, and compare a blind human subset
- Level lab connection: Lab 6 statistics and judge calibration
- Gate: Reproduce interval from raw records; explain why a significant but negligible effect may not justify release

Module 15 Application, retrieval, and agent evaluation

An application combines model behavior with retrieval, prompts, tools, policies, and user state. Evaluate retrieval recall and authorization separately from answer grounding and task completion. A correct answer drawn from another tenant's documents is still a failure. Agent evaluation must inspect sequences of decisions, tool arguments, side effects, retries, and stop conditions, not only the final message. Long-context tests should vary evidence position, distractors, conflicting instructions, and multi-hop dependencies. Tool tests need executable contracts and sandbox state assertions. Pair human or judge assessments with deterministic checks where possible, and distinguish schema validity from semantic correctness. Estimate

cost and time per completed task across the whole trajectory. A stronger model can produce a worse agent when it makes unnecessary calls, ignores policy boundaries, or becomes overconfident after a partial tool failure.

- Prerequisites: Module 14 and API architecture knowledge
- Suggested hours: 18
- Deliverable: Layered model/retrieval/application/agent suite with trajectories
- Bounded exercise: In two hours, classify ten seeded tool/grounding failures, move evidence across three context positions, and verify sandbox state separately from model claims
- Level lab connection: Lab 6 layered failure taxonomy
- Gate: Attribute ten failures to correct layers and independently verify side effects

Module 16 Safety, drift, and release qualification

Production qualification combines quality evidence with safety, privacy, reliability, and operational limits. Define unacceptable outcomes as explicit constraints instead of averaging them away inside a single score. Distinguish input distribution drift, label/concept drift, retrieval-corpus changes, behavioral regression, and evaluator drift. A changed input distribution is a reason to investigate, not proof that retraining is required. Compare cohorts and time windows with stable definitions, and account for delayed feedback. Safety evaluation needs representative misuse and accidental-failure scenarios, refusal quality, and authorized escalation paths. Assess long-context injection and tool boundary violations in controlled fixtures without exposing real systems. Release qualification must fail closed when provenance, sample size, or mandatory tests are missing. Define what can be approved, what must be rejected, and what remains inconclusive with a named decision owner.

- Prerequisites: Modules 14-15
- Suggested hours: 18
- Deliverable: Release gate policy and drift investigation matrix
- Bounded exercise: In 90 minutes, apply a predeclared policy to improvement, missing-evidence, and unauthorized-retrieval fixtures; define five drift hypotheses and their discriminating checks
- Level lab connection: Lab 6 PASS/FAIL/INCONCLUSIVE policy
- Gate: Aggregate gains cannot offset mandatory failures; missing evidence is non-pass

Lab 6 A release decision that can say inconclusive

Objective: Separate model/application/agent evidence while controlling comparison bias. Architecture: two releases answer 100 synthetic tasks; deterministic checks, blind humans, optional judge, and statistical layer yield a decision. Prerequisites: level 5 artifacts, locked split, SciPy, and ideally two reviewers. Solo mechanics do not establish independent reviewer agreement.

API: `scipy.stats.bootstrap((candidate, baseline), statistic, paired=True, confidence_level=0.95, n_resamples=9999, method="BCa", rng=42)` (C13). Statistic returns mean task-score difference. Align task IDs, not only lengths. Aggregate dependent turns per task or use cluster resampling. Handle missing values and degenerate intervals explicitly; never coerce them to a pass.

Measure correctness, retrieval recall, grounding, tool arguments, verified state, and end-to-end cost separately. Reverse A/B display on half the judge items. Place identical evidence early/middle/late. Seed a train/test duplicate, ten malformed tool calls, one unauthorized retrieval, and a shifted cohort. Blind-review a small subset and record disagreement without forcing consensus.

Expected output, unexecuted: paired interval, judge-human disagreement, failure taxonomy, PASS/FAIL/INCONCLUSIVE. An example lab policy requires a positive lower bound for improvement, complete provenance, and all privacy tests passing. These are chosen educational rules. Observe ordering mismatch, duplicate inflation, judge position bias, and aggregate gain hiding safety loss. Production equivalent: release-board evidence. Pass when every seeded integrity/policy fault blocks promotion.

Level 7 deployment

Module 17 Model artifacts and runtime selection

A deployable model is more than a weight file. Package configuration, tokenizer, chat template, generation defaults, adapters, quantization metadata, licenses, and evaluation evidence with immutable identities. Distinguish a registry's approval metadata from an object store's bytes and from a container registry's runtime image. Serving options occupy different layers: vLLM provides optimized LLM execution and APIs; Triton Inference Server manages model backends and scheduling; TensorRT-LLM targets NVIDIA LLM inference; ONNX is an interchange format with runtimes such as ONNX Runtime. TGI remains relevant to existing estates but its official documentation now says maintenance mode. Current TensorRT-LLM documentation also changes older engine-build assumptions. Compare model support, accelerator compatibility, debugging, upgrades, and reproducible performance before selecting a runtime. The decision must include an exit path, not merely the best advertised throughput figure.

- Prerequisites: Levels 5-6
- Suggested hours: 16
- Deliverable: Content-addressed model bundle and runtime decision matrix
- Bounded exercise: In 90 minutes, inventory one bundle, reject missing tokenizer/adaptor fixtures, and compare two plausible runtime candidates against the same acceptance matrix
- Level lab connection: Lab 7 immutable candidate and runtime prerequisites
- Gate: Reconstruct exact release; reject missing/incompatible components

Module 18 Inference scheduling and GPU performance

Inference performance depends on the request distribution and scheduling policy, not only GPU count. Prefill processes many prompt tokens together and often offers substantial compute parallelism; decode repeatedly reads weights and growing KV state for new tokens and can be bandwidth constrained. Continuous batching admits new work as other sequences finish. Paged KV allocation reduces fragmentation but does not remove the bytes required for useful context. Speculative decoding trades draft computation and verification for fewer target iterations; benefit depends on acceptance rate and workload. Tensor parallelism can improve fit or latency while adding communication. Measure time to first token, inter-token latency, output throughput, queue time, and goodput under an SLO. Study HBM bandwidth, FLOPs, PCIe/NVLink topology, and cache occupancy together before concluding that high utilization indicates efficient service.

- Prerequisites: Modules 7, 9, 17
- Suggested hours: 22
- Deliverable: Prompt/output-length load matrix and bottleneck diagnosis
- Bounded exercise: In two approved GPU hours, predict KV-limited concurrency, run a bounded subset of Lab 7's matrix, and compare memory fit with SLO-qualified throughput; no GPU means analysis only
- Level lab connection: Lab 7 load matrix and saturation boundary

- Gate: Explain predicted/measured limits; report goodput, not only raw tokens

Module 19 Serving contracts and delivery strategies

Online synchronous requests, offline batch jobs, asynchronous queued work, and streaming responses have different admission, retry, and cancellation contracts. Streaming can improve perceived responsiveness while leaving total completion time unchanged. Define bounded queues, deadlines, idempotency for side effects, and what happens when a client disconnects mid-generation. Separate rollout mechanics from experimental design: blue-green swaps environments, canary limits exposure, shadow duplicates selected traffic without returning candidate answers, and A/B tests compare randomized cohorts. Champion-challenger is a model-governance relationship that can use several delivery mechanisms. Keep stable capacity available for rollback and bind model, prompt, adapter, retrieval index, and runtime versions in one release. Tool side effects must never execute twice because of shadowing. Kubernetes Services alone do not provide precise weighted or mirrored traffic; use a verified routing integration when needed.

- Prerequisites: Modules 16-18
- Suggested hours: 16
- Deliverable: Serving API contract and blue-green/canary/shadow/A-B rollback plan
- Bounded exercise: In two hours, cancel one stream, saturate a bounded queue, and rehearse a read-only candidate cutback while checking no duplicate sandbox side effects
- Level lab connection: Lab 7 delivery and cancellation contract
- Gate: Bounded overload/cancellation and consistent identity without duplicate effects

Lab 7 Load-test an immutable serving candidate

Objective: Find the first limiting resource and validate rollback-safe serving. Architecture: load generator -> authenticated gateway -> stable/candidate vLLM; metrics and offline evaluation collect evidence.

Prerequisites: Linux, supported GPU, approved small bundle, pinned vLLM, private network. If only one engine fits, benchmark sequentially but leave simultaneous rollback-capacity evidence pending.

APIs: `GET /v1/models`, `GET /health`, `GET /metrics`, `POST /v1/completions`, `POST /v1/chat/completions` with a valid chat template (C14-C16). Later start `vllm serve` with immutable local model path and explicit context limits. This is an unexecuted HTTP sketch:

```
POST /v1/chat/completions HTTP/1.1
Content-Type: application/json

{"model": "lab-candidate", "messages": [{"role": "user", "content": "Summarize the supplied synthetic ticket."}], "max_tokens": 64, "temperature": 0, "stream": true}
```

Sweep prompts 128/1,024/4,096 tokens, output caps 32/256, concurrency 1/2/4/8 within safe memory. Separate cold/warm runs. Record actual lengths, warmup, offered/completed load, errors, queue time, first/inter-token latency, peak memory. Cancel a stream and verify resource release. Gateway request budgets protect administrative endpoints.

Expected output, unexecuted: performance matrix and capacity boundary, not a preset token rate. Introduce invalid template, oversize prompt, queue saturation, and quality regression. HTTP health can pass while quality fails. Shadow read-only tasks, cut back traffic, and drain. Production equivalent: runtime qualification and release rehearsal. Pass bounded overload, release identity, and declared recovery objective without assuming live KV migration.

Level 8 LLMOps

Module 20 Lifecycle lineage and reproducible releases

MLOps coordinates data, training, evaluation, and deployment. LLMOps extends the release identity to prompts, chat templates, retrieval indexes, quantization, judges, and model dependencies. AgentOps adds tool schemas, permissions, environment state, trajectories, and action policies. Treat these as overlapping operating responsibilities rather than three unrelated product categories. Create a dependency graph that connects every serving release to its data and evaluation evidence. CI validates deterministic contracts and artifact integrity; expensive statistical evaluations run in controlled pipelines; CD promotes only approved bundles. GitOps reconciles declared runtime state but cannot infer model quality from a manifest. Keep training credentials separate from release approval and serving identities. A rollback must restore compatible components, not just an earlier container image. Record intentionally nondeterministic elements and the limits of replay rather than promising impossible reproducibility.

- Prerequisites: Levels 6-7
- Suggested hours: 16
- Deliverable: Release manifest and data/model/prompt/tool/evaluator lineage graph
- Bounded exercise: In 90 minutes, trace two releases to immutable dependencies, remove one required digest/approval, and verify the promotion decision becomes non-pass
- Level lab connection: Lab 8 release registry and correlation identities
- Gate: Identify all upstream dependencies; missing mandatory digest/approval blocks promotion

Module 21 Observability and quality SLOs

Traditional request telemetry explains latency and errors but not whether an answer helped the user. Connect infrastructure metrics, inference scheduling, application traces, and delayed quality labels through bounded identifiers. OpenTelemetry provides transport and instrumentation patterns; GenAI semantic conventions are evolving and must be versioned rather than assumed permanently stable. Separate request availability, first-token latency, completion latency, grounded quality, and successful task completion into interpretable objectives. Define populations and measurement windows, including exclusions for invalid requests and missing labels. Token counts and cost are useful but do not measure correctness. Avoid high-cardinality prompt text in metric labels and avoid capturing sensitive content by default. Use sampled, authorized evidence when diagnosis needs text. Correlate a quality drop with prompt, retrieval, model, and runtime changes before treating it as a GPU problem.

- Prerequisites: Modules 15, 18, 20
- Suggested hours: 18
- Deliverable: Trace schema, privacy policy, dashboards, quality SLO definitions
- Bounded exercise: In two hours, correlate retrieval/generation/tool spans with one delayed label, specify missing-label denominators, and scan exports for a synthetic sensitive canary
- Level lab connection: Lab 8 telemetry and delayed quality join
- Gate: Locate a seeded regression without exporting raw confidential fixture content

Module 22 Feedback, incidents, and controlled improvement

Feedback is not automatically a training label. Users may abandon tasks, provide ambiguous ratings, or report outcomes long after generation; a feedback pipeline must preserve consent, provenance, and context. Separate an incident response from a model-improvement program. Rollback, traffic restriction, tool disablement, or human escalation may be safer than retraining during an outage. Route suspected drift through evidence collection and repeatable evaluation before changing datasets or rewards. Postmortems should connect failure mechanisms to preventive tests and owner actions without claiming one new test prevents all recurrences. Keep incident evidence out of training until privacy review and split rules permit reuse. Measure whether remediation improves the intended cohort and regresses another. Close the loop through an approved release, not through uncontrolled online learning.

- Prerequisites: Modules 16, 20-21
- Suggested hours: 14
- Deliverable: Feedback triage workflow, incident runbook, regression-test proposal
- Bounded exercise: In 90 minutes, classify five feedback records, contain one simulated incident without retraining, and propose a consent-reviewed regression test
- Level lab connection: Lab 8 incident diagnosis and rollback recommendation
- Gate: Correct feedback classification and no unauthorized model/data updates

Lab 8 Correlate a bad answer to a release

Objective: Join service traces, quality evidence, and controlled remediation. Architecture: gateway/application spans join retrieval, generation, sandbox tools, and delayed evaluation; collector exports redacted telemetry; registry resolves release identities. Prerequisites: level 7, telemetry sandbox, nonpersonal synthetic data, explicit retention.

APIs: `opentelemetry.trace.get_tracer`, `Tracer.start_as_current_span`, `Span.set_attribute`, `Span.record_exception`, and W3C `traceparent`. Configure an SDK provider/exporter later; the API alone is no-op. Use core tracing and owned attributes such as `arise.release.id` until a selected GenAI convention version is approved (C25-C26). OTLP is an integration boundary, not permission to export text.

```
gateway span
  application span [release id, approved task class]
    retrieval span [index digest, hit count, authorization result]
    generation span [model digest, token counts, timing]
    tool span [schema digest, policy result, sandbox outcome]
  later evaluation event [task evidence id, rubric version, result]
```

Change separately: missing retrieval document, prompt-template regression, slow tool. Hold weights constant. Show why model-version-only telemetry misses the first two. Join delayed labels through bounded keys; explain missing labels in SLO denominators. Metric dimensions use bounded release/task categories, not unique prompts/request IDs.

Expected output, unexecuted: three distinguishable traces, delayed join, privacy scan, rollback recommendation. Observe lost propagation, collector failure, evaluator drift, accidental text export. Telemetry failure must not block every inference. Production equivalent: cross-owner quality incident. Pass when evidence identifies the responsible component and canary content never appears in exported payloads.

Level 9 production platform

Module 23 Kubernetes GPU platform engineering

GPU scheduling requires compatible drivers, container runtime integration, and a device plugin or other supported resource allocation path. An operator can manage those components, but it does not erase upgrade compatibility or security review. Extended GPU resources are not ordinary CPU overcommit; inspect requests, limits, node labels, taints, tolerations, and topology. MIG partitions supported hardware, whereas time slicing shares a device without equivalent memory/fault isolation. Build separate policies for experiments, online inference, and distributed training. KEDA scales workloads from event metrics; the node autoscaler addresses unschedulable capacity, and neither makes model loading instantaneous. Weight caching, storage bandwidth, readiness checks, and graceful draining dominate many cold-start incidents. Measure a full path from requested replica to usable tokens, and ensure multi-GPU replica placement preserves the communication assumptions used in earlier benchmarks.

- Prerequisites: Modules 9, 18-21 and Kubernetes operations
- Suggested hours: 20
- Deliverable: GPU-pool/scheduling design and cold-start measurements
- Bounded exercise: In two hours on an approved sandbox, trace pod request through first token, introduce one taint/readiness fault, and distinguish workload from node scaling; stubs cannot prove GPU behavior
- Level lab connection: Lab 9 scheduling/startup waterfall
- Gate: Diagnose pending/driver/cache/readiness separately; time-slice count is not physical capacity

Module 24 Security, privacy, and tenant isolation

An LLM platform accepts untrusted data at multiple boundaries: training corpora, retrieved documents, prompts, tool results, checkpoints, and runtime packages. Separate data content from control authority, and enforce tool permissions outside the model. Protect corpus ingestion against poisoning with provenance, review, deduplication, and evaluation, while recognizing none proves a dataset harmless. Secure software/model supply chains with pinned artifacts, signatures, controlled builders, dependency review, and admission verification. Limit registry writes, training access, serving identities, and release approvals with distinct RBAC roles. Namespace separation alone is not a hard boundary for hostile tenants sharing a kernel or GPU. Include retrieval ACLs, cache keys, logs, quotas, and deletion workflows in tenant design. Privacy applies to prompts, responses, traces, embeddings, adapters, and backups, not only the source database.

- Prerequisites: Modules 15-16, 20, 23
- Suggested hours: 18
- Deliverable: Threat model with data/control boundaries and isolation tests
- Bounded exercise: In two hours, threat-model two synthetic tenants, test cross-tenant retrieval/cache/tool denial, and attempt admission of an unsigned fixture only in the owned sandbox
- Level lab connection: Lab 9 authorization/isolation track
- Gate: Cross-tenant fixtures fail closed; unapproved artifacts cannot reach serving

Module 25 Capacity, economics, and resilient placement

Capacity combines memory fit, token throughput, queue stability, latency, availability, and input/output-length distributions. Use Little's Law within a defined boundary, then load-test bursts rather than deriving tail guarantees from averages. Compare small-model routing, semantic caching, exact-prefix reuse, and batching with their quality/privacy costs. Multi-tenant fairness requires admission budgets and scheduling, not only namespace GPU quotas. Model cloud, on-premises, and air-gapped placement using utilization, staffing, support, power, egress, reserved headroom, and recovery capacity. Define RTO/RPO separately for artifacts, indexes, session state, and audit evidence. Replicated weights do not create warm failover. Validate degraded modes and recovery dependencies; avoid synchronous cross-region tensor parallelism as the default substitute for independent regional replicas.

- Prerequisites: Modules 18-19, 23-24
- Suggested hours: 18
- Deliverable: Capacity workbook, fictional-cost sensitivity, DR matrix
- Bounded exercise: In two hours, reconcile KV/throughput limits for three traffic scenarios, halve activity in the fictional cost model, and tabletop loss of one warm cell
- Level lab connection: Lab 9 recovery/queue evidence and engineering calculations
- Gate: Include failure headroom and defend placement across three demand scenarios

Lab 9 A GPU service survives platform friction

Objective: Validate pod scaling, node capacity, loading, and tenant boundaries. Architecture: private Kubernetes sandbox, online GPU pool, asynchronous queue, object-backed model cache, serving deployment, KEDA, telemetry. Prerequisites: reviewed Linux GPU cluster, administrator-approved device integration, level 8 artifacts, two synthetic tenants. Without GPUs use stubs for controller logic and mark GPU/MIG/performance gates pending.

APIs: `apps/v1 Deployment`, `v1 Service`, `v1 PersistentVolumeClaim`, `networking.k8s.io/v1 NetworkPolicy`, `rbac.authorization.k8s.io/v1 Role` and `RoleBinding`, `autoscaling/v2 HorizontalPodAutoscaler`, `keda.sh/v1alpha1 ScaledObject`. NVIDIA-specific illustrative fragment:

```
resources: {requests: {cpu: "2", memory: 4Gi}, limits: {cpu: "4", memory: 8Gi, nvidia.com/gpu: 1}}
```

This is not a complete deployment. Choose CPU/RAM from measurement, add toleration, affinity, immutable approved model, startup/readiness probes. GPU limit becomes request when request omitted; explicitly setting both requires equality (C22-C24). KEDA trigger measures actionable queue/service pressure, not token throughput alone. Preserve one warm interactive replica; scale to zero only if cold-start delay is acceptable.

Burst queued tasks, deny lab model-store access, evict one approved test pod. Record scheduling, image loading, weights, warmup, and first usable token separately. Tenant B's access to A's retrieval/cached answer must fail. Simulate restore from signed artifacts into a separate namespace or cluster; a namespace replay does not prove real regional DR.

Expected output, unexecuted: startup waterfall, bounded queue, isolation evidence, recovery timing. Observe taint mismatch, missing device allocation, early readiness, cache stampede, cold failover. Production equivalent: platform acceptance, not pod-readiness-as-service-readiness. Pass correct fault attribution, isolation, and declared recovery target or explicitly reject the design.

Level 10 principal architect

Module 26 Enterprise architecture and portfolio decisions

Principal-level design starts with business outcomes, risk appetite, demand, and constraints before selecting a model or platform. Translate those inputs into workload classes and explicit decision boundaries: buy versus build, shared versus isolated serving, adaptation versus retrieval, and managed versus self-operated infrastructure. Use a reference architecture to standardize identity, evidence, artifact flow, and telemetry while allowing runtime adapters where hardware or regulation differs. Compare portfolio reuse with coupling and shared failure domains. A standard model gateway can simplify policy but become a central outage path or hide provider-specific behavior. Quantify the cost of that abstraction and define escape hatches. Defend choices using measured workload envelopes and rejected alternatives, including simpler non-LLM solutions. Architecture quality is demonstrated by traceable decisions and controlled change, not by the number of boxes in a diagram.

- Prerequisites: Levels 1-9
- Suggested hours: 18
- Deliverable: Enterprise reference architecture and five ADRs
- Bounded exercise: In two hours, choose one workload, map each proposed component to requirement/owner/failure, and draft buy/build plus shared/isolated ADR alternatives
- Level lab connection: Lab 10 requirements envelope and reference architecture
- Gate: Every major component has requirement, owner, failure mode, measurable criterion

Module 27 Architecture assurance and organizational leadership

Architecture assurance asks whether the evidence is adequate for the consequence of failure. Establish decision rights across product, data science, platform, security, legal, finance, and operations without making every change depend on one committee. Define minimum release evidence, exception expiry, ownership, and escalation paths. Review uncertainty explicitly: model behavior, traffic forecasts, hardware supply, and evolving runtime support have different risk controls. Rehearse migration, incident containment, and disaster recovery with teams that own dependencies. Use measurable service-level/cost commitments. Mentor peers by asking for counterexamples and alternative explanations rather than rewarding agreement with a reference design. A principal architect also retires complexity and stops unjustified deployments. Distinguish reversible experiments from long-lived regulatory, data, or infrastructure commitments.

- Prerequisites: Module 26
- Suggested hours: 18
- Deliverable: Assurance checklist, ownership matrix, exception policy, review agenda
- Bounded exercise: In 90 minutes, assign approvers/evidence/reversal paths to five high-impact decisions and rehearse one expiring exception with a peer, or label review solo
- Level lab connection: Lab 10 independent-review and ownership requirements
- Gate: Independent reviewer can find approver, evidence, and reversal path for each decision

Module 28 Capstone design and evidence defense

Integrate the lifecycle into one bounded enterprise service rather than building a miniature foundation-model company. Choose a document assistant or supervised operations copilot with synthetic tenants, an approved small model, one adaptation experiment, one retrieval path, and sandboxed tools. Reuse earlier

evidence but check compatibility across the assembled release. Present model, application, and agent evaluations separately; reconcile memory, capacity, latency, and cost; then rehearse a failed candidate and recovery. The capstone includes written design and oral defense in which a reviewer changes a constraint. Explain what changes, what stays invariant, and what needs remeasurement. Success is an honest, reproducible decision package, including rejected features and unresolved risks. It is not production certification, and a supported no-go decision can satisfy the learning objective.

- Prerequisites: Modules 1-27; resolved or explicit hardware evidence gaps
- Suggested hours: 28
- Deliverable: Capstone package, demonstration record, architecture defense
- Bounded exercise: Use a three-hour review slice to present five evidence artifacts, rehearse rejected-release rollback, and respond to one unseen constraint; complete remaining package within module hours
- Level lab connection: Lab 10 and the unanswered capstone request below
- Gate: Meet rubric without undisclosed assumptions or unexecuted results described as measurements

Lab 10 Architecture review under a changed constraint

Objective: Defend evidence-backed service, then revise its envelope. Architecture: one deployment cell, one standby, two tenants, release registry, separate evaluation and serving identities. Prerequisites: earlier nine labs and ideally independent reviewer. A solo tabletop is not independent assurance.

API anchors: Lab 7 inference endpoints, Lab 9 Kubernetes APIs, OTLP, immutable OCI image references. The course-owned contract is a design sketch, not a standard:

```
{
  "release_id": "capstone-candidate-02",
  "model_digest": "required-sha256",
  "runtime_image_digest": "required-sha256",
  "tokenizer_digest": "required-sha256",
  "prompt_digest": "required-sha256",
  "retrieval_snapshot": "required-immutable-id",
  "tool_policy_digest": "required-sha256",
  "evaluation_digest": "required-sha256",
  "approval_state": "pending"
}
```

The digest strings specify required fields, not valid artifacts or filled evidence. Use genuine digests during implementation. Demonstrate a clean release, rejected release, and full-bundle rollback. A reviewer introduces no external network, doubled context, regional capacity loss, or hard tenant isolation. Before changing any running environment, produce revised memory/capacity, threat boundary, dependencies, and decision record.

Expected output, unexecuted: go/no-go decision, revised architecture, evidence-gap register. Observe stale costs, missing offline dependencies, incompatible retrieval rollback, insufficient warm reserve. Production equivalent: principal investment and release review. Pass internal consistency, named owners, and identified tests, without treating tabletop reasoning as runtime proof.

Engineering numerical models

Effective batch, steps, and loss normalization

For data-parallel degree D , microbatch size b per replica, and accumulation factor A :

$$B_{\text{effective}} = bAD$$

Tensor-parallel ranks cooperating on the same examples do not multiply batch again. For 10,240 examples, $b = 4$, $A = 8$, $D = 2$, batch is 64; a complete epoch has 160 updates and three epochs 480, assuming full groups and no dropped/padded examples. At 512 valid tokens/example this is 32,768 tokens/update; variable lengths invalidate that fixed conversion.

Equal averaging of microbatch means requires equal relevant counts. For variable language targets, aggregate summed token loss divided by all valid targets and account for distributed reduction scaling. Track examples, valid tokens, successful steps, skipped steps, and scheduler steps separately. Equal epoch labels do not imply equal optimization trajectories.

Weight and training-state memory

7B means 7,000,000,000 parameters; GB is 10^9 bytes and GiB is 2^{30} bytes. For a uniform b -bit representation the ideal weight payload is:

$$M_{\text{weights}} = P \frac{b}{8}$$

Parameters	FP32 GB	BF16/FP16 GB	INT8 payload GB	INT4 payload GB	BF16/FP16 GiB
7B	28	14	7	3.5	13.04
13B	52	26	13	6.5	24.21
70B	280	140	70	35	130.39

INT4/INT8 are packed payloads, not device-fit promises. Add scales, zero points where used, grouping, alignment, unquantized layers, adapters, conversion buffers, and any unpacked/extra representations. BF16 and FP16 use two bytes but differ in range. Use actual tensor counts, not rounded model names, for final planning.

Persistent full-training state can be inventoried as:

$$M_{\text{state}} = P(b_w + b_g + b_m + b_v + b_{\text{master}})$$

Each b is bytes/parameter for weights, gradients, two moments, and optional master copy. This is a ledger, not an optimizer law:

- FP32 parameters/gradients/two Adam moments: $4 + 4 + 4 + 4 + 0 = 16$ bytes/parameter
- Low-precision weights/gradients, FP32 moments and extra master: $2 + 2 + 4 + 4 + 4 = 16$
- Low-precision weights, FP32 gradients/moments and extra master: $2 + 4 + 4 + 4 + 4 = 18$

Parameters	16-byte state GB	18-byte state GB	16-byte GiB	18-byte GiB
7B	112	126	104.31	117.35
13B	208	234	193.72	217.93
70B	1,120	1,260	1,043.08	1,173.47

The Hugging Face guide describes an 18-byte arrangement (C03); PyTorch AMP creates default-precision parameters and autocasts operations (C02). Neither proves all Adam uses 16 bytes or all mixed precision keeps two weight copies. Dtypes depend on optimizer, precision, sharding, and configuration. AMSGrad adds state; quantized optimizers can reduce it; plain SGD needs less.

Add activations, temporary workspaces, communication buffers, allocator reservation, framework context, and checkpoint staging for peak memory. FSDP/ZeRO changes per-rank residency and introduces gather transients; do not divide every category by GPU count. LoRA retains the frozen base while trainable-state costs mostly scale with adapters and explicitly unfrozen parameters. For a d_{in} by d_{out} linear map, rank r adds $r(d_{\text{in}} + d_{\text{out}})$ parameters. At 4,096 by 4,096 with rank 8: 65,536 versus 16,777,216 full parameters. Activations remain separate.

KV cache for a concrete GQA decoder

For full-attention decoder layers with uniform retention:

$$M_{\text{KV}} = 2LH_{\text{kv}}d_h s \sum_{i=1}^C T_i$$

Two counts keys/values; L layers; H_{kv} KV heads; d_h head dimension; s bytes/element; T_i retained sequence tokens. Query heads cannot substitute for KV heads under GQA. At $L = 32$, $H_{\text{kv}} = 8$, $d_h = 128$, BF16 $s = 2$:

$$M_{\text{KV}/\text{token}} = 2 \times 32 \times 8 \times 128 \times 2 = 131072 \text{ bytes} = 128 \text{ KiB}$$

Define 128K as 131,072 tokens: one sequence needs 17,179,869,184 bytes = 16 GiB ideal KV. If 128K means 128,000 instead: 16.777216 GB = 15.625 GiB. Four independent fully occupied 131,072-token sequences require 64 GiB before weights/overhead. An 8,192-token sequence uses 1 GiB.

Reserve output within supported total context. Prefix sharing saves storage only for exact shared prefixes where policy permits. Paging changes granularity, not useful KV bytes. Sliding/hybrid attention, latent compression, quantized caches, offload, and eviction need model-specific formulas. TP can shard KV heads in some layouts, but some head-count/parallel-degree combinations replicate. Inspect per-rank allocation. Device budget includes weights + KV + workspace + other state + headroom. A fitting 7B weight file does not prove four long-context sessions fit. Startup KV-capacity logs are estimates, not arbitrary-traffic SLO guarantees (C14-C16).

Compute, bandwidth, and collective traffic

A first-order dense-transformer estimate is approximately $6PN$ training FLOPs for P parameters and N tokens when parameter matmuls dominate. It omits important attention, embeddings, recomputation, communication, and optimizer costs. Refine before long-context/MoE procurement. With achieved useful

aggregate throughput F_{eff} , time is approximately $6PN/F_{\text{eff}}$. Peak tensor FLOPs are not achieved throughput; match precision/sparsity and efficiency assumptions.

An operation with F FLOPs and Q moved bytes has ideal roofline time lower bound $\max(F/F_{\text{peak}}, Q/BW)$. Launches, synchronization, communication, and contention add time. HBM is not PCIe bandwidth; fast NVLink/NVSwitch does not imply equally fast inter-node links. A ring all-reduce of S bytes over D ranks transfers approximately $2(D-1)S/D$ per rank plus latency. NCCL may choose another algorithm. These explain trends, not exact unmeasured benchmarks.

Concurrency and Little's Law

For a stable system with consistent boundaries:

$$\bar{L} = \lambda \bar{W}$$

At 20 requests/second and mean end-to-end residence four seconds, mean in-system requests are 80. One second mean queue time and three seconds active service imply 20 queued and 60 active on average, not necessarily engine batch 80. Do not mix gateway rates with engine-only latency or use p95 instead of the mean.

At 400 output tokens/request this demands 8,000 output tokens/second plus prefill. Measure replica rate at matched lengths and latency target. Throughput and memory both lower-bound replicas; then add bursts, failure reserve, cold starts, and fairness. Average stability cannot bound tails. Arrivals above service cause queue growth. Agents amplify inference calls through retries/loops; forecast both tasks and calls.

Explicitly fictional illustrative cost model

Invented prices and performance for arithmetic, not GPU/provider quotes: four GPUs at \$2.50/GPU-hour for 720 hours/month; \$1,800 platform/storage/operations allocation; fleet output 1,000 tokens/second when active for 40 percent of the month. Variable charges are excluded only for this example.

$$\text{Cost} = 4 \times 2.50 \times 720 + 1800 = \$9000$$

$$\text{Tokens} = 1000 \times 0.40 \times 720 \times 3600 = 1,036,800,000$$

Allocated cost is \$8.68/million output tokens. If 90 percent is attributed to accepted successful tasks under this exercise's rule, useful output is 933.12 million, cost \$9.65/million useful output tokens. This is not cost per successful task. At 20 percent activity and fixed cost, raw output-token cost doubles to \$17.36/million. Real total cost needs training, failover reserve, egress, prompt work, evaluation, licenses, power, staffing, and failures. Token savings can increase task cost if they cause retries or worse answers.

Serving alternatives and lifecycle decisions

Alternative	Appropriate role	Important limitations	Course use
PyTorch + Transformers	Transparent reference execution and experiments	Not automatically admission/scheduling/fleet management	Numerical truth and adaptation baseline
vLLM	LLM execution, continuous scheduling, paged KV, compatible APIs, parallel serving	Qualify model/hardware/features; historical kernel descriptions differ	Default measured serving lab
Triton Inference Server	Multi-backend serving, ensembles, HTTP/gRPC, dynamic/sequence scheduling	Generic batching is not token-level scheduling; backend determines LLM behavior	Mixed ML/LLM estate comparison
TGI	Existing Hugging Face serving estates	Official maintenance mode, minor fixes/docs/light maintenance	Operation and migration, not default new investment
TensorRT-LLM	NVIDIA-focused LLM runtime/serving	Version/hardware/model coupling; current docs remove old TensorRT execution backend	Specialized alternative, no assumed speed superiority
ONNX + ONNX Runtime	Portable graph and execution-provider inference	Export/opset/shapes vary; InferenceSession is not a complete LLM service	Classifiers, encoders, qualified generation

Triton Inference Server is not the Triton GPU kernel language. Server and execution backend can coexist. Generic dynamic batching combines compatible stateless requests; iterative sequences allow stepwise scheduling for compatible backends, but retrieved docs label that feature provisional (C18). Verify exact versions/backends.

TGI docs explicitly recommend future engines including vLLM/SGLang and local options such as llama.cpp/MLX (C17). Maintenance mode does not mean current installations immediately fail; no security-support end date was established. Inventory models, templates, stops, logprobs, streaming, quantization, metrics, and performance before migration. Compare same bundle/traffic; API resemblance is insufficient.

The retrieved TensorRT-LLM migration page, updated 2026-09-04 at commit c295dd9, says PyTorch is now the sole execution backend, old `trtllm-build`/conversion is removed, and Hugging Face checkpoints load directly (C20). Do not mandate engine building for every current deployment. Older pinned releases can retain that workflow; match docs to installed version. This is a documented state, not a local compatibility test.

Choose through architecture, precision, context, adapters, structured outputs, stream cancellation, hardware, startup, memory, goodput, security, and upgrade acceptance criteria. Test a second candidate only when evidence could change the decision. Do not deploy six stacks to fill a checklist. Optional SGLang/local-engine comparisons need a separate compatibility investigation.

Enterprise reference architecture

Proposed cloud-neutral topology

OFFLINE CONTROL AND EVIDENCE PLANE

Sources --> [1 Intake / rights / privacy] --> [2 Versioned corpora]
[2] --> [3 Training / adaptation] --> [5 Candidate artifact store]
[2] --> [4 Retrieval snapshots]
[CI builders / SBOM] --> [5]
[5] --> [6 Independent evaluation / humans] --> [7 Release registry]
[7] --> [8 GitOps / admission / deployment] --> signed serving bundle

ONLINE DATA PLANE

Clients --> [9 Identity / gateway / admission] --> [10 Application / agent]
[10] --> [11 ACL retrieval] --> [4 Retrieval snapshot]
[10] --> [12 Model router] --> [14 Serving cell A / GPU replicas / KV]
[12] --> [Exact response cache] --> cache miss to [14]
[12] --> [Async queue] --> [14]
[10] --> [13 Tool policy broker] --> [Sandbox / approved APIs]
[Model cache / immutable weights] --> [14]
[9 Gateway failover] --> [15 Independent cell B / warm reserve]
[8 Approved bundle] --> [14] and [15]

FEEDBACK AND OBSERVABILITY PLANE

All components --> [16 OTel / metrics / audit / quality joins]
[16] --> [17 Incident and feedback triage]
[17] --> reviewed data to [1], never directly to training

Interaction contracts and ownership

1. Intake admits permitted sources, records rights/provenance, and quarantines suspicious/sensitive records. Data owners authorize use; ingestion is not a training-rights grant. Retention, deletion, identity, and export constraints travel with data.
2. Versioned storage records immutable manifests and transforms. Training/retrieval can derive different products. Split before transforms that could leak family information. Mutable URLs are insufficient to reconstruct experiments.
3. Training reads approved corpora and writes candidates, state, and evidence. Training identities cannot approve production. Workers use trusted private fabric and controlled checkpoint/rendezvous access.
4. Retrieval snapshots bind ACLs, embedding identity, chunking, indexes, freshness. Check caller authorization before assembling context. Rollback must respect current deletion/access requirements, not restore deleted privileges.
5. Artifact storage preserves weights, adapters, tokenizer, template, configuration, quantization, runtime references. CI builds reviewed images and SBOMs. Signatures establish identity/provenance, not behavioral safety; evaluate the candidate.
6. Independent evaluation consumes candidates and protected holdouts, emitting raw records, statistics, evaluator identity, and human outcomes. Errors are non-pass. Training cannot access private benchmark answers through shared credentials.
7. Registry binds immutable artifacts to approvals and supported workload envelopes. Approval names evidence, owner, expiry/review conditions. Serving resolves a bundle, not a floating latest-model label.
8. Deployment reconciles approved state; admission enforces artifacts/policy. Rollout coordinates routing through verified integrations. GitOps convergence is not quality readiness, which needs independent release/runtime checks.

9. Gateway authenticates, establishes tenant, validates limits, enforces deadlines and quotas before expensive work, and propagates traces/cancellation. It needs its own availability design, not one unprotected cross-region dependency.
10. Application assembles prompts, retrieval, calls, and tools under bounded budgets. Preserve session release identity or manage transitions explicitly. Agent state records observed tool outcomes, not only generated claims of success.
11. Retrieval returns authorized evidence/source identities. Distinguish source content from executable authority. Freshness, recall, permissions, and grounding have separate monitors/owners.
12. Router selects approved models by workload/tenant policy, not only cost. Exact response cache, semantic cache, and prefix-KV reuse have different correctness guarantees. Keys include tenant, release, authorization, corpus/template identity. Async queues preserve deadlines/idempotency.
13. Tool broker authorizes outside the model and mediates sandbox/approved APIs. High-impact actions need approvals/audit. Shadow requests never execute business effects; generated arguments remain untrusted until validated and authorized.
14. Serving cells admit bounded work with known weights/cache budgets. Local caches verify digests and avoid unbounded concurrent downloads. Separate training bursts from interactive pools. A TP/PP group is one coordinated replica for scaling/failure, not several independent HTTP pods.
15. Standby/active second cells load compatible approved artifacts and reserve capacity. Failover covers gateway/DNS, identity, retrieval, queues, and audit, not only weights. In-flight KV need not usually replicate; define application restart/resume behavior.
16. Telemetry uses bounded metadata, explicit sampling/retention, and controlled content access for delayed quality joins. Utilization, pressure, first-token latency, task quality, and privacy violations remain separate signals.
17. Incident/feedback triage separates containment from learning. Evidence becomes corpus/test data only after consent, privacy, and split review. No direct production-feedback-to-weight-update path. Product/data/platform/security owners decide the action.

Cloud, on-premises, and air-gapped estates implement the same contracts with different identity, storage, registries, GPU pools, and networks. No provider-specific deployment is prescribed. Air gaps require mirrored weights, tokenizers, packages, images, trust roots, licenses, documentation, and recovery dependencies before disconnection, plus controlled updates/evidence export. No internet is not a complete supply-chain control.

Ten unanswered architecture case prompts

For each case, list unknowns, propose your design/hypothesis, specify the smallest discriminating experiment, and name evidence that would reverse your decision. The reference architecture is context, not a case-specific answer. Submit your proposal before requesting critique; no solutions are supplied here.

1. A domain adapter reduces training loss by 35 percent but user task success does not improve. What would you inspect before acquiring data or increasing rank? Design a comparison separating objective, data, and application effects.
2. A 7B model's weights fit comfortably, yet four long-context sessions cause OOM. Which measurements/dimensions are missing? Design bounded admission experiments.
3. Benchmark gain is two percentage points, candidate answers are longer, and the judge favors first position. How would you determine meaningful gain without tuning the final holdout?

4. Two nodes double GPUs but reduce throughput. Which hypotheses distinguish shapes, fabric, rank participation, and data loading? Specify evidence before redesigning the topology.
5. A TGI estate serves five families with undocumented stops. How would you decide whether/how to migrate given maintenance mode? Define equivalence and rollback.
6. KEDA adds pods during bursts but first tokens take minutes. What boundaries would you instrument? Separate workload scaling, node provisioning, and loading ownership before proposing a fix.
7. Tenants share a semantic cache; a correct answer cites another tenant's document. What does this imply about acceptance/isolation? Propose an evidence-backed redesign.
8. GRPO reward climbs but humans see repetitive low-information responses. How would you distinguish rewards, generation, implementation defaults, and task gain?
9. Finance requests 40 percent lower cost while operations requires unreduced regional failover. Which demand, quality, staffing, and reserve assumptions must be tested before committing? Present alternatives, not an unsupported promise.
10. A regulated customer requires offline operation in six weeks; tokenizer/docs download at startup and evaluation uses an external judge. What evidence must an architecture review require before approving an air-gapped design?

Capstone request and acceptance rubric

Prepare a two-tenant enterprise knowledge/operations assistant decision package using synthetic/permitted documents and sandbox tools. Choose one measurable task, one retrieval path, one adaptation comparison, one candidate serving stack, and a non-LLM or prompt-only baseline. This is an engineering demonstration, not an autonomous production operations agent. Propose the design; no solution is supplied.

Submit requirements envelope, data/model cards, lineage manifest, architecture diagram, five ADRs, paired evaluation, capacity/memory workbook, load-test record, privacy/isolation checks, release/rollback evidence, DR tabletop, and unresolved risks. Separate measurements, estimates, and expected results in every artifact. Identify reused evidence and what needs rerunning after dependency changes.

Criterion	Points	Observable requirement
Problem and baseline	10	Task/users/constraints, simpler baseline, no-go conditions
Data and adaptation	15	Permitted provenance, split integrity, comparison, retention
Evaluation	20	Model/application/agent separation, raw paired evidence, uncertainty, safety gates
Runtime and capacity	15	Memory ledger, matched load, goodput, reserve
Operations and recovery	15	Immutable release, correlated telemetry, rejection, rollback, DR limits
Security and privacy	15	Tenant tests, tools, artifacts, retention boundaries

Criterion	Points	Observable requirement
Decision defense	10	Rejected alternatives, owners, changed constraints, honest gaps

Educational pass: at least 80/100 plus mandatory integrity. No unauthorized data, hidden test leakage, unexecuted results passed off as measurements, cross-tenant fixture disclosure, or production side effects. Explicit missing physical distributed evidence prevents validated multi-GPU claims, not all architecture work. A justified no-go can pass. Independent review remains pending if only a solo tabletop is possible.

After defense, choose a focused second iteration, such as long-context goodput under a fixed quality margin, two-rank scaling failure, or runtime migration. Repeated bounded iterations build depth more effectively than repeating the whole survey curriculum.

Topic coverage matrix

Coverage means a planned instructional home and evidence path, not completed mastery. Module 1 alone is fully taught here; later detail expands in the live A-L sequence.

Required topic family	Modules	Practical evidence / lab	Current depth
Data-to-feedback lifecycle; modality-specific representation	1, 5, 20-22	Lifecycle map; Labs 1/8	Full first lesson plus roadmap
Vectors, matrices, loss, gradients, chain rule, splits	1-2	Hand/autodiff trace; Labs 1/2	Full M1; networks briefing
Networks, activations, initialization, CNN/RNN biases, regularization	2	Linear/MLP shapes, learning curves; Lab 2	Briefing/exercise; advanced modalities follow-on
SGD/momentum/Adam/Ada mW, learning rate/warmup/decay, epochs/steps/batch	1, 3	Update ledger and sweep; Labs 1/2	Worked mechanics plus roadmap
Accumulation, valid-token normalization, AMP/FP16/BF16, checkpointing	1, 3-4	State/dtype/resume ledger; Labs 1/2	Formula/first lab plus GPU extension
Tokenization, embeddings, labels, templates, masks	5	30 strings/two templates; Lab 3	Briefing and API specification
QKV/GQA, causal attention, RoPE, RMSNorm/LayerNorm, FFN, logits	6	Exact shape and causal tests; Lab 3	Shapes specified; implementation deferred
Generation/sampling, prefill/decode, context/KV	7, 18	Cached parity and load matrix; Labs 3/7	Formula/briefings

Required topic family	Modules	Practical evidence / lab	Current depth
Pretraining/continued pretraining, data rights/dedup, mixtures/curricula, synthetic data/scaling	8	Corpus/token/compute budget; Lab 4	Briefing/specification
SFT/RLHF/DPO/GRPO, rewards, reward hacking	10	Preference/group reward worksheet; Lab 4	Objective contracts; optional training
DP/TP/PP/SP/CP/EP, FSDP/ZeRO, MoE, collectives	9, 18, 23	Partition/fabric map and two-rank check; Labs 4/7/9	Hardware evidence pending
GPU HBM/FLOPs/bandwidth, PCIe/NVLink/InfiniBand/RoC E	4, 9, 18	Memory/roofline/traffic ledgers	Analytical, no performance claims
Full FT/LoRA/QLoRA, bottleneck/prefix/prompt/IA 3, quantization	11-13	Counts, identity/retention and compatibility; Lab 5	Briefings and supported API sketches
Model/application/agent evaluation, human/judge/pairwise, statistics	14-15	Paired interval, blind rubric, trajectories; Lab 6	Specification; sample calibration pending
Leakage/contamination, safety/drift, context/tool evaluation	14-16	Integrity and boundary fixtures; Lab 6	Controlled-sandbox specification
Registry/artifacts, vLLM/Triton/TGI/TensorRT-LLM/ONNX	17	Bundle/runtime acceptance; Lab 7	Current-source caveats preserved
Online/batch/async/streaming; continuous/paged/speculative inference	18-19	Cancellation/queue/load matrix; Lab 7	Performance experiments pending
Blue-green/canary/shadow/A-B/champion-challenger/rollback	19-20	Full-bundle rollout rehearsal; Labs 7/10	Verified routing integration still required
MLOps/LLMOps/AgentOps, data/prompt/eval versioning, CI/CD/GitOps	20-22	Dependency graph and gate; Lab 8	Briefings/specifications
OTel, end-to-end traces, quality SLOs, delayed labels	21	Privacy-safe quality join; Lab 8	Schema/version caveat retained
Poisoning/supply chain/privacy, IAM/RBAC/tools/tenant isolation	16, 24	Threat model and fail-closed fixtures; Labs 6/9	Defensive sandbox, not security certification

Required topic family	Modules	Practical evidence / lab	Current depth
Kubernetes plugins/operators, taints, MIG/time-sharing, KEDA/node scaling, storage/caches	23	Startup waterfall and isolation; Lab 9	Linux GPU execution pending
Capacity/economics/routing /caches/fairness; cloud/on- prem/air gap/DR	25-27	Numerical workbook, DR dependencies, ADRs; Labs 9/10	Fictional prices; measured inputs required
Enterprise components, ownership, assurance, build/buy	26-28	Seventeen-component contracts and capstone	Design request, not solved cases
Practice per module and progressive assessment	All 28	28 bounded exercises; ten cumulative labs; M1 bank/quiz	Later 27 banks and full executable expansions deferred

Course resources and verified free-access catalogue

Sixteen curated resources support selective study, not sixteen mandatory complete courses. Free content is not free compute, credit, or certification. Thirteen distinct YouTube endpoints were identity-checked in the source research; three more Karpathy links were recovered from his syllabus without separate destination checks. CS229 has a verified official MP4 alternative. No video was watched end to end; playlist identity does not verify every entry, region, or current playback.

Verification labels: Y = official source plus direct YouTube identity; L = exact YouTube link on official source only; M = official non-YouTube media verified; C = public course/offer verified, gated lessons not audited; Conditional = advertised limited-time free offer, not permanent access. Evidence is from the 2026-09-13 research; browser event logs showed 2026-09-12 UTC, so no exact wall-clock verification time is claimed. Synthesis did not refetch external pages.

Corrected course-to-level mapping

Level	Objective	Resources	Required evidence / boundary
L1	Numerical learning and generalization	R01, R02 micrograd, R03	Single-step trace and split rationale
L2	Networks, optimization, precision	R02, R03, R04	Correct training loop and state ledger
L3	Tokens, transformers, generation	R02, R05, R06	Shapes/masks/cache parity
L4	Pretraining, distributed GPU systems, post-training objectives	R07, R08	Corpus/compute/fabric budget and reward worksheet

Level	Objective	Resources	Required evidence / boundary
L5	Fine-tuning and adaptation	R06, R09, selected R07	Base/tuned comparison and adapter identity
L6	Model/application/agent evaluation	R07, R09 evaluation units, R10/R11 testing	Paired evidence, calibration, release gate
L7	Inference and serving	R07, R10, selected R11	Measured workload envelope, rollback
L8	MLOps/LLMOps, observability, CI/CD	R10, R11, R12	Lineage, feedback, monitoring
L9	Kubernetes, SRE, defensive security	R13, R14, R15	Controlled rollout, SLOs, authorized sandbox
L10	Principal architecture	R16, R10, R14	Capacity/failure/cost/ownership ADRs

Distributed GPUs are L4, fine-tuning is L5, evaluation is L6 throughout this plan. Videos support concepts; current runtime configuration, GPU scheduling, enterprise IAM/supply chain, and incident command need the primary documents and later labs.

R01 Stanford CS229 Machine Learning public archive

- Creator: Stanford Engineering Everywhere, Andrew Ng; identity confirmed by official Lecture 1 transcript
- Official source: [CS229 public archive](#)
- Media: [Official Lecture 1 MP4](#); later downloads/transcripts available from archive; no exact YouTube link exposed by inspected page
- Verification: M; course and browser anchors inspected, MP4 HEAD 200 with `video/mp4`; [transcript](#) fetched; playback not tested
- Mapping/order: L1. Refresh probability/linear algebra, then introduction/supervised learning, learning theory/generalization, selected unsupervised learning; RL optional before L4 post-training
- Exercise: NumPy regularized regression/classification baseline with independent train/validation/test and bias/variance/leakage diagnosis
- Cost/age: Public materials, not university credit. Historical MATLAB/Octave should be translated to Python. Restricted Summer 2026 resources are not substituted for this archive.

R02 Neural Networks Zero to Hero including micrograd

- Creator: Andrej Karpathy
- Official source: [Zero to Hero syllabus](#)
- Start: [Backpropagation: building micrograd](#)
- Next exact syllabus links: [Building makemore](#), [Let's build GPT](#), [GPT Tokenizer](#)
- Verification: Y for micrograd, matching browser title/player; L for three next links, present with descriptions on official syllabus but not independently opened

- Mapping/order: M1 numerical support, L2-L3, bridge to L4. Python/elementary derivatives first; micrograd, makemore, intervening MLP/activation/backprop lessons in syllabus order, GPT, tokenizer. Do not skip tensor foundations.
- Exercise: Scalar autodiff with finite differences, reproduce in PyTorch, then tiny character decoder with masking/tokenization tests
- Cost/age: Public videos; CPU-scale experiments possible, no entitlement to large-model compute. Instructional GPT is not a current distributed trainer or production ChatGPT replica.

R03 Practical Deep Learning for Coders 2022

- Creator: Jeremy Howard, fast.ai
- Official source: [fast.ai course](#)
- Video: [Practical Deep Learning for Coders playlist](#)
- Verification: Y; official free statement and link, matching browser title, HTTP 200
- Mapping/order: L1-L2 with early deployment preview. Coding prerequisite; select lessons 1/2 for motivation, then Part 1 gaps alongside R02 when high-level APIs hide mechanics. The plan remains concept-first even when this resource demonstrates tool-first learning.
- Exercise: Modest transfer-learning classifier, split/error analysis, high-level fastai versus plain PyTorch loop
- Cost/age: Free videos/book; historical Kaggle/Paperspace offers are not verified 2026 quotas. Version-check fastai, Gradio, notebooks, deployment APIs. Optimization and feedback principles remain useful.

R04 Stanford CS231n Deep Learning for Computer Vision 2025

- Creator: Stanford CS231n team; current site names Fei-Fei Li, Ehsan Adeli, Justin Johnson and colleagues, not necessarily every archived lecturer
- Official source: [CS231n](#)
- Video: [2025 recording playlist](#)
- Verification: Y; official prior-recording link; HTTP 200 title identifies Stanford CS231N Deep Learning for Computer Vision I 2025, not 2026
- Mapping/order: L2, optional CV depth. Python/calculus/linear algebra/probability; early classification/loss/optimization/backprop, then training/convolution topics, preserving selected playlist order
- Exercise: Overfit a tiny image batch, diagnose gradient/initialization fault, compare regularization on held-out data
- Cost/age: Public archive, not current Canvas access/credit/student GPU credits. Match assignments to offering and framework version rather than mixing current syllabus with old recordings.

R05 Stanford CS224N NLP with Deep Learning Spring 2024

- Creator: Stanford, Christopher Manning for verified 2024 playlist
- Official source: [CS224N](#)
- Video: [Spring 2024 playlist](#)
- Verification: Y; official page calls complete 2024 set free; browser HTTP 200 title confirms year/instructor; Winter 2026 recordings restricted to enrolled students

- Mapping/order: L3, conceptual support for L5-L6. After R02/basic ML, study word representations/language modeling before attention/transformers/pretrained models; select rather than duplicate mastered fundamentals
- Exercise: Compare embeddings, encoder classifier, and causal decoder on distinct tasks; explain objectives, masks, and evaluation
- Cost/age: Public recordings/slides differ from paid XCS224N certificates/credit. New 2026 syllabus topics are not promised in 2024 videos. APIs/rankings/reasoning techniques age faster than NLP concepts.

R06 Hugging Face LLM Course

- Creator: Hugging Face course team, including Sylvain Gugger, Lewis Tunstall, and named collaborators
- Official source: [LLM Course introduction](#)
- Video: [Hugging Face Course playlist](#)
- Verification: Y; source says completely free without ads, links playlist; HTTP 200 title matches. Current written syllabus extends historical videos; complete parity not checked.
- Mapping/order: L3 and L5; evaluation portions support L6. Python/introductory DL prerequisite. Chapters 1-4 for Transformers/Hub/FT, 5-8 for datasets/tokenizers/tasks, selected advanced 10-12 later; defer demo-sharing chapter 9 until needed.
- Exercise: Small encoder fine-tune; compare `Trainer`/manual loop and retain model/tokenizer/data revision/evaluation
- Cost/age: Free course, account needed for Hub publishing; GPUs/endpoints may cost. Fetched FAQ says no certificate. Prefer maintained Transformers/Datasets/Accelerate contracts when recordings disagree.

R07 Stanford CS336 Language Modeling from Scratch Spring 2026

- Creator: Percy Liang and Tatsunori Hashimoto, Stanford
- Official source: [CS336 current course](#)
- Video: [Spring 2026 Lecture 1 with course playlist](#)
- Verification: Y; official schedule/link and HTTP 200 title Spring 2026 Lecture 1: Overview, Tokenization. Not evidence every later recording is complete.
- Mapping/order: L4 pretraining/distributed/post-training, support L5-L7. Fluent Python/PyTorch/ML/memory hierarchy/calculus/probability. Lectures 1-4 tokenization/accounting/architecture; 5-8 accelerators/kernels/parallelism; 9-14 scaling/inference/evaluation/data; 15-17 post-training/alignment.
- Exercise: Scaled-down official Basics and Systems assignments; state/activation budget and data-cleaning comparison before large training
- Cost/age: Public lectures/repositories; multi-GPU and assignment infrastructure not guaranteed free/external. CPU correctness first. Dated course GPU prices are not current quotes. Pin assignments and verify kernels/parallel APIs.

R08 GPU MODE profiling and collective communication selections

- Creator: GPU MODE; repository names Lecture 16 Taylor Robbie and Lecture 17 Dan Johnson
- Official source: [Lecture repository](#), linking the [GPU MODE channel](#)
- Videos: [Lecture 16: On Hands Profiling](#), then [Lecture 17: NCCL](#)

- Verification: Y; repository confirms identities; linked official-channel search supplied exact anchors; both destination visits HTTP 200 with matching titles, NCCL associated code/slides
- Mapping/order: L4, reinforced L7. Complete CS336 accounting/basic GPU parallelism, then profiling before NCCL to give communication measurable context
- Exercise: Profile one step, separate compute/synchronization, compare predicted/measured scaling; without multi-GPU, analyze a supplied trace and derive all-reduce budget, explicitly not a benchmark
- Cost/age: Public lectures; hardware and recommended PMPP book separate. CUDA/PyTorch profiler/NCCL settings version-sensitive. Triton kernel language differs from Triton Inference Server.

R09 Hugging Face smol-course for fine-tuning

- Creator: Hugging Face, Ben Burtenshaw and collaborators
- Official source: [smol-course onboarding](#).
- Media: Public course units; no exact lesson/playlist YouTube URL exposed by inspected onboarding, so none is invented
- Verification: C; public text confirms free course, prerequisites, instruction tuning/evaluation/preference alignment/VLMs, and advertised free certification rules; activities/certificate issuance not tested
- Mapping/order: L5 fine-tuning, L6 evaluation, with L4 post-training reinforcement. Transformer/Python/PyTorch prerequisites. Unit 1 instruction tuning/templates, Unit 2 domain evaluation, Unit 3 preference alignment; delay multimodal/RL extensions until baseline evaluation stable.
- Exercise: Base versus SFT domain suite, templates, contamination, regressions, memory; add PEFT variant using current guidance
- Cost/age: Free account/course and advertised certificates, optional paid Pro/GPU. Later October/November unit notices lack reliable year context; completeness not promised. TRL/PEFT/quantization contracts evolve.

R10 Full Stack Deep Learning 2022

- Creator: FSDL team; selected deployment lecture by Josh Tobin
- Official sources: [2022 course](#) and [Lecture 5 Deployment](#)
- Video: [Lecture 05: Deployment](#)
- Verification: Y; official free lecture/lab statement, HTTP 200 HTML embed exposed exact ID with playlist `PL1T8f07ArWleMMI8KPJ_5D5XS1ovTW_Ur`; equivalent watch URL independently returned matching title/200
- Mapping/order: L7-L8/L10, testing supports L6. After a trained model: lectures 1 when ML, 2 infrastructure, 3 testing, 4 data, 5 deployment, 6 continual learning; revisit 8 teams/9 ethics for architecture. Only Lecture 5 individual endpoint checked.
- Exercise: Separate model/UI lifecycles, batch versus request-time serving, gradual rollout/rollback, feedback/tests
- Cost/age: Free material does not guarantee hosted labs; W&B/cloud/API/GPU conditions separate. 2022 CPU/serverless preferences are not universal autoregressive-LLM advice. Deployment vendors/frameworks may have changed.

R11 Full Stack Deep Learning LLM Bootcamp Spring 2023

- Creator: FSDL; selected LLMOps lecture by Josh Tobin

- Official sources: [Spring 2023 bootcamp](#) and [LLMOps lecture](#)
- Video: [LLMOps](#)
- Verification: Y; free course, HTTP 200 HTML exposed embed ID with playlist `PL1T8f07ArW1eyIq0y370VXsP4hFXymd0Z`; equivalent watch URL independently returned matching title/200. Other lecture scopes verified by page, not each endpoint.
- Mapping/order: L7-L8/L10, application testing supports L6. After R06, Foundations recap, Prompt Engineering, Augmented Language Models, LLMOps, askFSDL walkthrough
- Exercise: Retrieval application evaluation set, versioned prompts/retrieval/models, categorized errors, feedback-informed gate
- Cost/age: Free recordings, not current sponsor/API/vector/GPU credits. Task evaluation/licensing/versioning endure; 2023 GPT-4/Claude rankings and LangChain/hosting examples are historical.

R12 DataTalksClub MLOps Zoomcamp

- Creator: DataTalks.Club; Cristian Martinez, Alexey Grigorev, Emeli Dral listed instructors
- Official source: [MLOps Zoomcamp repository](#)
- Video: [MLOps Zoomcamp playlist](#)
- Verification: Y; repository free-course/exact-link evidence; browser 200 matching title and backlink
- Mapping/order: L8, monitoring bridge to L9. Python/Docker/CLI/ML prerequisites. Modules 1-2 maturity/tracking, 3 orchestration, 4 deployment, 5 monitoring, 6 testing/CI/laC, project; skip already-mastered tracking details.
- Exercise: Version data/models, register/test/deploy locally, Prometheus/Grafana operational metrics and Evidently data/drift report; drift is not proven quality loss
- Cost/age: Free self-paced; repository says no 2026 live cohort and no self-paced certificate. Cloud may charge. Pin coherent MLflow/Prefect/Evidently editions rather than mixing refreshed pages and old videos.

R13 Introduction to Kubernetes LFS158

- Creator: The Linux Foundation
- Official sources: [Course offer](#) and [course portal](#)
- Media: Portal course/video demonstrations; no exact YouTube playlist discovered
- Verification: C; public offer confirms architecture, Minikube, building blocks, authn/authz/admission, services, volumes, ConfigMaps/Secrets/Ingress; \$0, login/enrollment, digital badge, 90 days access. Gated lessons not tested.
- Mapping/order: L9. Linux/containers first; chapters 2-5 architecture, 6-9 cluster/building blocks, 10-15 access/network/deployment/config. Experienced operators use a gap check.
- Exercise: Local CPU model service, config/credentials separation, rollout/rollback, access boundaries; GPU scheduling/isolation requires additional material
- Cost/age: Advertised \$0 badge is not CKA/CKAD. No claim of mandatory paid edX certificate. Local labs avoid cloud fees, not hardware costs. Match controllers/manifests to selected Kubernetes.

R14 Google The Art of SLOs

- Creator: Google Customer Reliability Engineering; companion explainer Riccardo Carlesso
- Official source: [The Art of SLOs workshop](#)

- Video: [The Art of SLOs](#)
- Verification: Y; public slides, participant/facilitator handbooks, worksheets, CC-BY-4.0; HTML 200 supplied exact link, matching browser title/200
- Mapping/order: L9-L10 after deployment/monitoring. Watch explainer, slides, attempt participant exercises, then compare facilitator responses. Companion video is not the whole workshop.
- Exercise: Separate availability/TTFT/completion/task-quality SLIs, denominators, windows, budgets, escalation ownership; do not compress all into one score
- Cost/age: Public video/workshop; linked Coursera/certificate not included or verified free. Calibrate objectives to actual economics, not sample percentages.

R15 Red Teaming LLM Applications conditional free option

- Creator: DeepLearning.AI with Giskard; Matteo Dora and Luca Martial
- Official sources: [Public course offer](#) and [introductory lesson](#)
- Media: No exact full-course YouTube link discovered; a promotional trailer is not substituted
- Verification: C, Conditional; public page lists seven video lessons/five code examples and limited beta free access; graded accomplishments marked Pro. Enrollment/playback untested.
- Mapping/order: L9 after Python and R11 evaluation context. Vulnerability overview, manual testing, scaling, LLM-assisted tests, full assessment, only in an owned/authorized isolated sandbox
- Exercise: Define defensive scope, assess instruction/data-boundary failures in an isolated fixture, record mitigations, add safe regression cases; no third-party production testing
- Cost/age: Recheck limited-time free offer before assignment; accounts/API/Pro separate. Not comprehensive IAM, supply-chain, privacy, or Kubernetes security training. Integrations and coverage evolve.

R16 Google SRE Classroom Distributed ImageServer

- Creator: Google Site Reliability Engineering
- Official sources: [ImageServer workshop](#) and [official recording short link](#)
- Video: [SRE Classroom: Design a Distributed System in One Hour](#); discovered [mobile destination](#)
- Verification: Y; public page links slides/recording, short-link HEAD resolves to mobile YouTube/200, browser canonical page matches title/200. Source linkage explains title difference.
- Mapping/order: L10 after R14/distributed systems. Read requirements, propose your own design, then watch worked workshop and compare NALSD workbook. Title is not a mastery-time promise. This does not supply answers to the ten course-specific cases.
- Exercise: Transfer capacity/failure reasoning to serving replicas, memory, queues, redundancy, backpressure, build/buy ADR; revisit FSDL teams/ownership
- Cost/age: Public CC-BY-4.0 material; optional cloud/Kubernetes implementation not automatically free. Replace historical figures with measured data. Original workload is images, not an LLM course.

Ranked watch sequence

Exit by evidence, not video completion. Select relevant material and skip familiar software-architecture introductions, not ML evaluation or resource accounting.

Rank	Resource and route	Completion evidence
1	R01 supervised/generalization selections	Baseline, objective, split, failure analysis
2	R02 micrograd/makemore fundamentals	Hand/numerical/autodiff gradient agreement
3	R03 lessons 1-2 and Part 1 gaps	Train/expose small useful model
4	R04 optimization/training; CV optional	Gradient and overfitting diagnosis
5	R02 GPT/tokenizer then R05 transformer selections	Tokens, attention, mask, objective
6	R06 chapters 1-8; advanced later	Model/tokenizer/data/eval identities
7	R07 lectures 1-8 and small Basics/Systems	Decoder and measured resource ledger
8	R08 profiling then NCCL, at L4	Compute/memory/communication distinction
9	R07 scaling/data/post-training/eval; R09 units 1-3 across L4-L6	Base/tuned paired comparison and contamination controls
10	R07 inference; R10 testing/data/deployment	Capacity benchmark and rollback
11	R11 augmented models/LLMOps	Application tests and prompt/retrieval versions
12	R12 tracking/monitoring/CI/project selections	Operational/data monitoring and gate
13	R13 Kubernetes gaps	Local rollout/rollback
14	R14 explainer/workshop	SLIs, error budget, stakeholder agreement
15	R15 only after free-access confirmation	Authorized assessment and regression fixtures
16	R16 NALSD plus R10 teams/ethics	Capacity/risk/cost/ownership ADR

Resource verification evidence ledger

The catalogue preserves creator, exact publisher/media URL, prerequisite, sequence, exercise, cost, age, and access evidence for all sixteen entries. Publisher evidence was not inferred from search snippets. Direct checks covered these thirteen distinct endpoints, not every playlist entry:

Resource	Exact checked endpoint	Observed identity
R02	micrograd	Spelled-out intro to neural networks/backpropagation: building micrograd

Resource	Exact checked endpoint	Observed identity
R03	fast.ai playlist	Practical Deep Learning for Coders
R04	CS231n playlist	Deep Learning for Computer Vision I 2025
R05	CS224N playlist	NLP with Deep Learning I Spring 2024, Christopher Manning
R06	HF playlist	Hugging Face Course
R07	CS336 Lecture 1	Spring 2026 Overview, Tokenization
R08	Profiling	Lecture 16: On Hands Profiling
R08	NCCL	Lecture 17: NCCL
R10	Deployment	Lecture 05: Deployment (FSDL 2022)
R11	LLMOps	LLMOps (LLM Bootcamp)
R12	MLOps playlist	MLOps Zoomcamp
R14	SLO explainer	The Art of SLOs (Service Level Objectives)
R16	ImageServer recording	SRE Classroom: Design a Distributed System in One Hour

All except micrograd's initial open explicitly recorded HTTP 200; micrograd returned matching title/player without separately recorded status. FSDL IDs came from official privacy-enhanced iframe embeds, then destination checks, not guessed video IDs. The three extra R02 links remain L, not Y. R01's MP4 was checked with HEAD, not watched. R09/R13/R15 account-gated experiences remain untested.

Access distinctions retained: CS229 current/restricted pages are not the SEE archive; CS224N 2026 is not the public 2024 playlist; CS231n linked recordings are 2025; CS336 [2025 archive](#) is separate from its current 2026 schedule. A failed CS229 candidate path was excluded, not offered as a learner link. The [SRE Classroom overview](#) supported workshop scope; extra workshops and GPU MODE videos were not added to inflate coverage. No runtimes, timestamps, complete-playback claims, or unverified playlist IDs are manufactured.

Primary-source register

The following freely accessible project/author documents support mechanisms and API contracts. Vendor documents define vendor behavior, not independent performance rankings. Research date is 2026-09-13; synthesis preserves these findings without a new external verification pass. Moving aliases and unexecuted examples are not a version lock. C IDs are curriculum sources; S IDs are Module 1 sources; R IDs above are learning resources. Same URLs can appear in different namespaces intentionally.

Curriculum sources C01-C27

ID	Primary source	Supported claim and limit
C01	PyTorch AdamW	Decoupled decay, state APIs, foreach peak memory; stable alias redirected to 2.14
C02	AMP examples	Autocast/scaler, accumulation, unscale-before-clip, skipped steps; unexecuted examples
C03	HF memory anatomy	Weights/moments/gradients/activation s/temporaries; example precision layout not universal
C04	Transformers Llama	GQA config, RMSNorm/RoPE/SwiGLU, logits/cache; model-specific
C05	Chat templates	Training generation-prompt setting, duplicate special tokens; inspect actual template
C06	Compute-optimal training	Model/token scaling under compute budget; not universal lifecycle-cost optimum
C07	PyTorch TP tutorial	TP/SP, DTensor, mesh/FSDP composition, fast links; snippets require version-matched mesh validation
C08	NCCL collectives	Collective semantics and compatible rank participation; algorithms/topology selected at runtime
C09	TRL DPO	Preference fields and standard reference-relative objective without explicit reward model; options evolve
C10	TRL GRPO	Group generations/rewards/advantages/variants; defaults differ from original assumptions
C11	PEFT quantization	QLoRA/config/k-bit preparation/targets/merge caveats; hardware untested
C12	PEFT IA3	Activation-scaling vectors/frozen base; quality task-dependent
C13	SciPy bootstrap	Paired resampling/BCa/RNG/degenerate intervals; cluster design needed for dependence

ID	Primary source	Supported claim and limit
C14	vLLM parallelism/scaling	TP/PP, KV capacity logs, topology, private network warning; no local benchmark
C15	vLLM online serving	Completion/chat/health/metrics, template needs, administrative endpoint warning
C16	Historical paged attention	Blocked KV concept; explicit warning that historical kernel no longer matches current code
C17	TGI docs	Explicit maintenance mode and successor guidance; no support end date established
C18	Triton batchers	Dynamic versus stateful/iterative; iterative feature provisional
C19	TensorRT-LLM docs	Serving/KV/parallel/API/model support entry points, no throughput ranking
C20	TensorRT backend migration	Removed old execution/engine-build workflow; 2026-09-04, commit c295dd9
C21	ONNX Runtime Python	Export/check/InferenceSession/run; not full generative serving guarantee
C22	Kubernetes GPUs	Drivers/plugins, extended resources, request/limit equality, labels/affinity
C23	NVIDIA GPU sharing	MIG memory/fault isolation versus time-slicing limits; select hardware/version later
C24	KEDA v2.18 deployment scaling	Activation/HPA/termination; page marks 2.18 nonlatest and points to 2.20
C25	OTel GenAI move notice	Previous location no longer maintained, directs to separate repository
C26	GenAI conventions repository	New home; retrieved README has unfinished schema URL and no published releases; no stable schema guarantee
C27	Argo Rollouts traffic management	Native Service limits, routing/mirroring integrations, stable/canary; router-dependent support

C01-C03 ground training state; C04-C05 transformers; C06-C10 training; C11-C12 adaptation; C13 statistics; C14-C21 serving; C25-C26 observability; C22-C24/C27 platform/scaling/delivery. Hours, margins, architecture boundaries, and fictional costs are recommendations or derivations, not source-provided performance

claims.

Module 1 sources S01-S10

ID	Primary source	Supported mechanism
S01	Autograd mechanics	Graph/saved tensors/leaf accumulation/grad modes/mode separation
S02	Optimizer documentation	Parameter ownership, state, step order, conventions
S03	zero_grad	None versus zero semantics
S04	Adam	Moments, bias correction, coupled decay
S05	AdamW	Decoupled decay and state/memory
S06	AMP examples	Scaling/nonfinite skip/unscale/clip/accumulate
S07	Reproducibility	RNGs/determinism/workers/portability limits
S08	Saving/loading	Model/optimizer state, copied best-model state, loading modes
S09	Installation selector	Compatible CPU installation follow-up; selector not exercised
S10	Zero to Hero syllabus	Free micrograd/Python/calculus prerequisites; R02 supplies video and verification boundary

PyTorch stable URLs returned redirect stubs and explicit 2.14 pages were retrieved in source research. This identifies documentation served, not an installed or recommended tested version. Module 1's program is original explanatory code, not a copied course assignment. The old vLLM distributed URL yielded no useful content; C14 replaced it. Compatible-server redirected to C15. Redirect recoveries are not extra independent supporting sources.

Research evidence summary and validation boundaries

Research status: Complete for synthesis of the learning plan and first lesson. Lab execution, learner assessment, and hardware validation remain uncompleted. Handbook PDF and matching HTML are published with sampled visual validation. Publication does not establish runtime lab success or all-page accessibility.

The four comprehensively read inputs are provenance records, not competing versions of the learner plan. This primary document governs sequence, terminology, and scope:

- .copilot-tracking/research/subagents/2026-09-13/curriculum-architecture-research.md
- .copilot-tracking/research/subagents/2026-09-13/single-training-step-research.md
- .copilot-tracking/research/subagents/2026-09-13/free-ml-llm-video-courses-research.md
- .copilot-tracking/research/subagents/2026-09-13/learning-plan-preparation-research.md

Research questions resolved

Question	Selected answer and evidence
Sequence for an experienced architect?	28 modules/ten levels, concept-first with gates; platform expertise reconnects after mechanics/evaluation
What changes in one training step?	Module 1 A-L separates graph, gradients, parameters, optimizer, continuation state; S01-S08
Inspectable numerical example?	Two examples, two parameters; exact MSE 477/800 after simultaneous SGD update
Practical route for every module?	28 bounded exercises with outputs, ten cumulative specifications, one complete independent program
Which public courses and media?	R01-R16 catalogue with verified identity/access boundaries and corrected L4/L5/L6 mapping
How to account for hardware/economics/security?	Explicit memory/KV/compute/queue/cost models, controlled labs, architecture/ownership contracts
How to preserve publication readiness?	One authoritative source; published PDF/HTML with reproducible checks and sampled visual evidence

Evidence classes and dates

- Numerical evidence: source research used exact `fractions.Fraction` arithmetic for initial predictions 0.5/1.0, loss 5.625, gradients -7.5/-4.5, updated 1.25/0.45, predictions 1.7/2.95, and MSE 477/800 = 0.59625. This is independent arithmetic, not captured PyTorch output. Other resource/cost tables are analytical models under their stated assumptions, not measured GPU behavior.
- Code evidence: source research parsed the embedded program with Python AST. Syntax validity is not runtime, dependency, convergence, or checkpoint validation. PyTorch was absent from the selected application environment. No training, GPU profiling, model download, package installation, container build, or cluster operation is claimed here.
- Documentation evidence: C01-C27 and S01-S10 record primary-document observations dated 2026-09-13. TGI maintenance, TensorRT-LLM migration, KEDA version warnings, and GenAI convention relocation are time-sensitive; recheck before live labs.
- Media evidence: R01-R16 preserve publisher and direct-page observations, not complete playback, enrollment, certification, or permanent free access. Exercises are proposed applications unless explicitly identified as official assignments.
- Publication evidence: local MarkdownIt, pinned KaTeX, and Chromium produced a 68-page PDF. HTML preflight found 106 math expressions, zero KaTeX errors, zero targeted horizontal overflows, loaded fonts, and no missing anchors. Bounded stdlib extraction confirmed selectable prose/code and 19 distinct YouTube URLs in 31 URI annotations, not 19 unique videos or playback checks. PDF.js samples

covered equations, code pages 20-24, roadmap/serving tables, ASCII architecture, capstone, references, and ending. This is sampled proof, not inspection of every page or full mathematical copy/accessibility fidelity.

Publication source and outputs:

- .copilot-tracking/research/2026-09-13/ml-llm-lifecycle-learning-plan-research.md
- .copilot-tracking/research/2026-09-13/ml-llm-lifecycle-learning-plan.html
- .copilot-tracking/research/2026-09-13/ml-llm-lifecycle-learning-plan.pdf

Exact final bytes, SHA-256, page-count stability, sample pages, and limitations: .copilot-tracking/research/subagents/2026-09-13/final-handbook-publication-research.md. The render-only command omits the optional verifier; it was not repaired. Assets, profiles, caches, and intentional outputs stay under research; no installs occurred.

Synthesis validation

Saved-file checks on 2026-09-13 confirmed sequential 28-module, ten-level, and ten-lab inventories; 28 bounded exercises with deliverables/gates/lab connections; 486 core hours with the stated per-level totals; A-L coverage; the exact 5/5/5/5/3/3/2 assessment distribution; five quiz questions; nine first-lab fields; ten glossary terms; ten unanswered cases; sixteen resources; and 27 C plus ten S primary references. The 253-line embedded program parses, and its executable AST matches the researched program after excluding docstring whitespace.

Standard-library checks independently reconfirmed the exact single-step arithmetic, KV payload, weight/state GiB tables, LoRA counts, batch/token counts, reserve hours, and fictional cost calculations. Markdown structure, metadata, JSON and GPU YAML were parsed; headings, fences, source namespaces, external links, and math escapes were checked. No PyTorch code was imported or run. Syntax/structure checks do not establish runtime correctness or rendered mathematical layout. Publication places YAML frontmatter first so the verified renderer strips it rather than displaying metadata in the body; the obsolete leading lint comment has been removed.

Remaining coverage limits

Later 27 A-L lessons, their individual 28-prompt banks/quizzes, and fully expanded executable nine-field labs are intentionally deferred. Every module already has a practical exercise and deliverable; that does not make each a full runnable lab. All ten cumulative labs remain unexecuted, including the complete first program. No hardware benchmark, independent human review, security certification, current application capability, or production readiness is inferred.

Classical baselines and CNN/RNN biases provide context. Advanced vision, speech, multimodal internals, research specialization, deeper enterprise IAM/supply-chain practice, and incident-command experience need follow-on depth, not another hidden level. No hardware means distributed/MIG/performance evidence stays pending. Sample sizes and safety margins need capstone-specific calibration. Free-media access does not fund compute. No legal/jurisdiction-specific deployment approval is supplied.

Selected approach and actionable next steps

Use the measured concept-first sequence. Begin with Module 1 Session 1, keep the ten-term glossary, predict the arithmetic, and submit answers before feedback. Progress after evidence-based gates, not watch time. Expand one later A-L module at a time; reuse its bounded exercise in the cumulative level lab. Complete evaluation before any external-user deployment and keep tools in an authorized controlled sandbox. Treat a well-supported no-go as legitimate engineering work.

Follow-up scope

- [] Pin and record a compatible Python/PyTorch CPU lab environment without changing application dependencies; later select permitted model revisions and a coherent Transformers/PEFT/TRL/runtime matrix.
- [] Execute the standalone Module 1 program, capture actual assertions/logs/versions, and investigate any difference before claiming runtime success.
- [] Extend checkpoint validation to durable storage and a new process before claiming crash recovery.
- [] Measure available hardware and supported Linux/GPU access; set a spend ceiling using real quotes, not the fictional cost model; mark multi-GPU gaps explicitly.
- [] Expand subsequent A-L lessons and nine-field executable lab instructions as reached; use bounded datasets, pinned versions, and retained failure/recovery traces.
- [] Calibrate evaluation sample sizes, human/judge agreement, mandatory safety coverage, and release margins for the chosen capstone.
- [] Recheck conditional R15 free access before assignment; choose a permanently public replacement if expired. Check R09/R13 account conditions if certificates/access matter.
- [] Recheck TGI lifecycle, TensorRT-LLM workflow, KEDA release, and GenAI conventions when implementing, not by assuming moving docs remain unchanged.
- [] Map the cloud-neutral architecture to a named estate only after identity, jurisdiction, network, and recovery constraints are known.
- [x] Publish PDF and matching HTML; record actual PDF text/links, sampled math/code/table/layout inspection, and accessibility limits in the final publication report. All-page visual and assistive-technology audits remain unperformed.

Personalization questions that do not block the plan

How comfortable are you with matrix calculus/probability and reading PyTorch? Is the 8-10 hour weekly budget realistic? Is one GPU or a two-GPU Linux environment available, and what is the monthly spend ceiling? Is the target role applied AI, inference platform, or training infrastructure architect? Which workload/jurisdiction should shape the capstone? Can a peer independently review the rubric and defense, and do certificates matter? Until answered, retain small-model, CPU-first, synthetic-data, cloud-neutral assumptions and all not-yet-assessed labels.

[Back to contents](#)